

Formal Verification Report: Morpho Midnight

- Date: June 11th, 2026
 - Audit Repo: <https://github.com/alexzoid-eth/morpho-midnight-fv>
 - Client Repo: <https://github.com/morpho-org/midnight>
 - Audit Commit: 7538c438513622721e23a94676b93a335b83dace
 - Mitigation Commit: —
 - Author: [AlexZoid](#)
 - Certora Prover version: 8.13.0
-

Table of Contents

1. [About Midnight](#)
 2. [Formal Verification Methodology](#)
 - [Verification Approach](#)
 - [Types of Properties](#)
 - [Verification Process](#)
 - [Assumptions](#)
 3. [Verification Properties](#)
 - [Valid State](#)
 - [High-Level](#)
 - [Market Creation](#)
 - [State Transitions](#)
 - [Reachability](#)
 - [Reverts](#)
 - [Access Control](#)
 - [Gates](#)
 4. [Real Issues Found](#)
 5. [Verification Results](#)
 6. [Setup and Execution](#)
 - [Common Setup \(Steps 1-5\)](#)
 - [Remote Execution](#)
 - [Local Execution](#)
 - [Running Verification](#)
 7. [Resources](#)
-

About Midnight

Midnight is a fixed-rate, fixed-term lending protocol built around zero-coupon **markets**. A market is a tuple `(loanToken, maturity, collateralParams[], gates, rcfThreshold)`: lenders hold tradable `credit` units redeemable for one loan-token each at maturity, and borrowers hold `debt` units owed to the market pool. Trading happens off-chain via signed `offer`s submitted on-chain to `take`, which mints/burns paired credit and debt against a maker/taker. Collateral is supplied

per-position (up to 128 distinct collateral tokens per market, with 16 active per borrower), health-checked by oracle-driven LLTV tiers; unhealthy or post-maturity positions are liquidated with a Liquidation Incentive Factor (LIF) that ramps post-maturity. Bad debt is socialized to lenders via a `lossFactor` slashed lazily at the next position interaction.

Settlement fees are charged per `take` (piecewise-linear interpolation by time-to-maturity, stored in centi-basis-points); continuous fees are time-accrued per-position (fixed at offer time). A multi-token `flashLoan` primitive is supported, multicall via self-delegatecall is allowed, and the market struct is stored as runtime bytecode at a CREATE2 address derived from its id (SSTORE2-style data store). Offers can only be placed at ticks that are multiples of the market's tick spacing. The design borrows isolated-pool, LLTV-tier, and Aave-style loss-socialization ideas from Morpho Blue, adding (a) tradable order-book semantics, (b) pluggable `enterGate` / `liquidatorGate` per-market access modules, and (c) pluggable `IRatifier` callbacks gating offer acceptance.

The formal verification scope covers a single target:

1. **Midnight** (`src/Midnight.sol`, ~1015 lines) — the sole core contract; holds every position, every market-state slot, and every loan-token / collateral-token / fee-claim balance routed through `safeTransfer` / `safeTransferFrom`. All entry points (`take`, `withdraw`, `repay`, `supplyCollateral`, `withdrawCollateral`, `liquidate`, `flashLoan`, `multicall`, `claimContinuousFee`, `claimSettlementFee`, `updatePosition`, `touchMarket`, plus role / fee / tick-spacing admin, `enableLltv` / `enableLiquidationCursor`, and `setIsAuthorized` / `setConsumed`) are parametric in the run. Storage compartments verified: `position[id][user]`, `marketState[id]`, top-level mappings (`consumed`, `isAuthorized`, `defaultSettlementFeeCbp`, `defaultContinuousFee`, `claimableSettlementFee`, `isLltvEnabled`, `isLiquidationCursorEnabled`), role addresses (`configurator`, `feeSetter`, `feeClaimer`, `tickSpacingSetter`), and the transient `LIQUIDATION_LOCK_SLOT`. Auto-included pure-helper libraries: `UtilsLib`, `IdLib`, `TickLib`, `SafeTransferLib`, `EventsLib`, `ConstantsLib`. Periphery (`TakeAmountsLib`, `ConsumableUnitsLib`), authorizers (`EcrecoverAuthorizer`), and ratifiers (`EcrecoverRatifier`, `SetterRatifier`) are out of scope — their effect on Midnight goes through Midnight's public interface, which is verified directly.

Formal Verification Methodology

Certora Formal Verification (FV) provides mathematical proofs of smart contract correctness by verifying code against a formal specification. Unlike testing and fuzzing which examine specific execution paths, Certora FV examines all possible states and execution paths.

The process involves crafting properties in CVL (Certora Verification Language) and submitting them alongside compiled Solidity smart contracts to a remote prover. The prover transforms the contract bytecode and rules into a mathematical model and determines the validity of rules.

Verification Approach

Midnight is verified standalone — one Certora target, one harness ([MidnightHarness](#)), four conf files spanning two complementary market regimes (one-market vs many-market), and a single per-rule callbacks-loaded driver per regime. The verification splits the market-state surface into two narrowings — a single-market narrowing and a three-market narrowing (`idA`, `idB`, `idC`) — to keep the SMT problem tractable while still covering the cross-id reasoning that single-market rules cannot express.

- **One-market regime** ([midnight_valid_state_one.spec](#) + [setup/midnight_one.spec](#)) — the per-market ghost surface is collapsed to scalar (no-id) ghosts; `touchMarketCVL` pins all touched markets to a shared `loanToken`, and Sload mirror checks on per-market slots make paths that touch a second distinct id infeasible. This is where every per-position bound, every per-market conservation property, and every loan-token-side ERC-20-backing invariant lives.
- **Many-market regime** ([midnight_valid_state_many.spec](#) + [setup/midnight_many.spec](#)) — per-id ghosts are retained; a three-market narrowing (`ghostMiMarketIdA`, `ghostMiMarketIdB`, `ghostMiMarketIdC`) restricts the prover to three distinct touched markets. This regime carries the full valid-state invariant set expressed as explicit conjunctions over the three narrowed markets (`idA`, `idB`, `idC`), with the ERC-20-backing invariants as explicit sums over them.

External dependencies are modelled in CVL: ERC-20 tokens via balance / allowance ghosts ([erc20.spec](#) + [safe_transfer_lib.spec](#)) with no callbacks; the oracle via a positive-price ghost ([oracle.spec](#)); the gates as NONDET booleans ([gates.spec](#)); the ratifier and take/buy/sell/repay/liquidate/flash callbacks as `CALLBACK_SUCCESS`-returning summaries with empty data ([ratifier.spec](#), [callbacks.spec](#)); CREATE2-based `IdLib.storeInCode` via a non-zero-address ghost and `IdLib.toId` via a parameterised hash ghost ([id_lib.spec](#)); `UtilsLib` arithmetic (`mulDivDown` / `mulDivUp`), bit-counting, and transient storage as CVL summaries ([utils_lib.spec](#)); `TickLib.tickToPrice` as a monotone five-tick ghost model ([tick_lib.spec](#)).

Together, the four runs prove every per-market valid-state invariant under the single-market narrowing, and the same set enumerated over a three-market narrowing where the ERC-20-backing invariants sum over multiple markets.

Types of Properties

Properties are categorized following the [official Certora methodology](#). Valid State, Variable Transitions and State Transitions properties are **parametric** -- they are automatically verified against every external function in the contract, including functions added after the specification is completed. High-Level properties target specific function sequences.

Valid State -- System-wide invariants that MUST always hold true. These properties define the fundamental constraints of the protocol, such as accounting consistency and structural integrity. Once proven, these invariants serve as trusted assumptions in other properties via `requireInvariant`, reducing verification complexity.

High-Level -- End-to-end behavioral guarantees on specific entry points and call sequences.

State Transitions -- Multi-variable co-transition properties: how two or more storage variables must change together (or one's change forces another's) across a single call.

Reachability -- `satisfy()` rules proving that meaningful non-reverting execution paths exist from a valid state. Their purpose is anti-vacuity: they confirm that the narrowed verification model still reaches the dangerous-but-required states (unhealthy positions, bad-debt realisation, credit/debt minting, full exits) that the other rule categories depend on.

Reverts -- `@withrevert` rules proving that functions revert under disallowed conditions (the dual of reachability): unauthorized callers (anti-theft), invalid inputs, and broken state preconditions.

Access Control -- parametric rules asserting that a role-gated or authorization-gated state variable changes only when the caller holds the role or is authorized (state-change $\Rightarrow$ role). The robust dual of the revert rules: this ghost-state-change form catches any function that writes the gated variable — including a renamed or second setter that a selector-anchored revert rule would miss. The remaining categories (Variable Transitions, Unit Tests, ERC-4626 Conformance) are out of scope for this report.

Verification Process

1. **Setup phase:** Define ghost variables, storage hooks, and helper definitions to model contract state in CVL. Establish the verification harness (`MidnightHarness`) and conf files for each regime. This phase also addresses several prover limitations:
 - ERC-20 token model (`erc20.spec`, `safe_transfer_lib.spec`): replaces `IERC20.transfer` / `transferFrom` and `SafeTransferLib.safeTransfer*` with CVL summaries that mutate ghost balance / allowance maps exactly by the requested amount and revert on insufficient funds. No reentrancy back into Midnight is modelled (per the protocol's TOKEN REQUIREMENTS).
 - Oracle model (`oracle.spec`): `IOracle.price()` returns a per-oracle ghost constrained to the well-behaved positive branch.
 - Gate callbacks (`gates.spec`): `canIncreaseCredit`, `canIncreaseDebt`, `canLiquidate` are NONDET; the prover explores both `allow` and `veto` branches.
 - Ratifier and entry-point callbacks (`ratifier.spec`, `callbacks.spec`): summarised to return `CALLBACK_SUCCESS` with empty payloads, matching the way `take`'s `require(... == CALLBACK_SUCCESS)` prunes every non-success branch.
 - IdLib summaries (`id_lib.spec`): `IdLib.toId` is a parameterised hash ghost wired to market field tuples; `IdLib.storeInCode`'s CREATE2 deployment is collapsed to a non-zero-address ghost.
 - UtilsLib summaries (`utils_lib.spec`): `mulDivDown` / `mulDivUp` use the standard `int / int` quotient; `countBits` / `msb` / `setBit` / `clearBit` use compact ghost models; transient `tExchange` / `tGet` are wired to a CVL ghost.
 - TickLib summaries (`tick_lib.spec`): `tickToPrice` is narrowed to a five-tick monotone model so price/tick arithmetic stays in scope.
 - `Midnight.touchMarket` (`setup/touch_market_summary.spec`): summarised via `touchMarketCVL` (already-touched markets only). `touchMarket` is public, so the `internal` summary intercepts the external ABI entry too — creation is dead code wherever the summary is loaded; the creation branch is therefore verified in its own conf, `market_creation.conf` → `midnight_market_creation.spec`, which imports the base setup without the summary.
2. **Crafting Properties:** Write invariants and rules in CVL, starting with valid state invariants (which become trusted preconditions for other rules), then state/variable transitions, and finally high-level and unit test properties. Variable transitions, unit tests, and ERC-4626 conformance are out of scope for this engagement.

Assumptions

Formal verification requires assumptions about the code and its environment to address prover timeouts, tool limitations, and state consistency. However, incorrect assumptions can mask real bugs by excluding reachable states from analysis. To maintain transparency, all assumptions are categorized into four groups: **Safe** (real-world constraints that don't reduce security coverage), **Proved** (formally verified invariants reused as preconditions), **Unsafe** (scope reductions necessary for tractability that may exclude valid scenarios), and **Trusted** (initialization state and admin-configured parameters assumed correct because initialization logic is excluded from verification).

Safe Assumptions

These reflect real-world constraints that don't impact security guarantees. In the codebase, every `require` statement that constitutes a safe assumption is annotated with a `"SAFE: ..."` message string.

Environment Constraints ([env.spec](#)):

Rule `env` is constrained to physically realisable values — non-zero block number and timestamp within a plausible 32-bit window. The same-`env` helper enforces that any auxiliary `env` (e.g. the parametric function `env` in a preserved block) shares block, timestamp, sender, and value with the rule `env` so identity-on-time properties hold across nested calls.

- `e.msg.value == 0` — no ETH transferred to Midnight (Midnight is not payable).
- `e.msg.sender != 0` and `e.msg.sender != currentContract` — sender is a non-zero, non-self EOA / external contract.
- `e.block.timestamp` is bounded in `[max_uint16, max_uint32)` — realistic timestamp window.
- `e.block.number != 0` — non-zero block number.
- Preserved-block `env` matches the rule `env` in `block.number`, `block.timestamp`, `msg.sender`, `msg.value`.

ERC-20 Compliance ([erc20.spec](#), [safe_transfer_lib.spec](#)):

The CVL ERC-20 model encodes the protocol's TOKEN REQUIREMENTS — total supply equals the sum of all balances, transfer/transferFrom move exactly the requested amount, decimals fall within a realistic range, and `address(0)` cannot be a token / counterparty.

- Total supply equals the sum of all balances (no rebase, no fee-on-transfer).
- Token decimals between 6 and 18 (covers every common ERC-20).
- `called contract != address(0)` on every transfer/transferFrom/approve/balanceOf path.

Market Creation Validity ([midnight.spec](#)):

The `touchMarketCVL` summary checks the exact set of conditions that the real `touchMarket` enforces at the moment of creation — collateralParams non-empty, sorted strictly by token address, with each collateral's `lltv` / `liquidationCursor` validated by the numeric consequences of the runtime creation gates (LLTV tiers and liquidation cursors are now governance-enabled at runtime via `enableLltv` / `enableLiquidationCursor`, not a fixed set). The transient liquidation lock starts unlocked because the slot does not persist across transactions.

- `collateralParams.length > 0` (matches `NoCollateralParams()` revert).
- `collateralParams[0].token != 0` (matches `CollateralParamsNotSorted()` for index 0).
- `collateralParams[1].token > collateralParams[0].token` when `length > 1` (sort invariant).
- `validCollateralParamsCVL(lltv, liquidationCursor)` on each collateral: `lltv <= WAD` (mirrors `LltvNotEnabled/enableLltv`), `liquidationCursor < WAD` (mirrors `LiquidationCursorNotEnabled/enableLiquidationCursor`), `maxLifCVL(lltv, cursor) <= 2*WAD` (mirrors `InvalidMaxLif`), and `lltv == WAD || lltv*maxLif <= 0.999e18*WAD` (mirrors `MaxLifTooHigh`).
- Transient `LIQUIDATION_LOCK_SLOT[id][user]` starts unlocked at the top of every rule.

Oracle ([oracle.spec](#)):

The oracle is modelled on its well-behaved positive branch — a misbehaving oracle returning zero reverts on the protocol's downstream math, and that revert behaviour is itself a documented LIVENESS consideration rather than a state-correctness concern.

- `ghostMiOraclePrice256[o] >= 1` for every oracle address.

Proved Assumptions

These properties are verified as valid state invariants and are used as trusted preconditions (via `requireInvariant`) in the preserved blocks of other valid-state invariants. See [Valid State](#) for detailed descriptions.

Setup re-entry ([midnight_valid_state_one.spec](#)):

The one-market valid-state setup carries every PASS'ed invariant forward into each rule's preserved block via `requireInvariant`, so any function that may have been weakened by an earlier rule still sees a tight pre-state. The full list is built in `setupValidStateOneMidnight`.

- [VS-MI-01](#) `creditCoversPendingFee`
- [VS-MI-02](#) `positionLastLossFactorWithinMarket`
- [VS-MI-03](#) `lastAccrualNotInFuture`
- [VS-MI-04](#) `collateralBitmapMatchesSlot`
- [VS-MI-05](#) `nonEmptyPositionImpliesTouched`
- [VS-MI-06](#) `creditAndDebtMutuallyExclusive`
- [VS-MI-07](#) `creditOrLastLossFactorImpliesLastAccrual`
- [VS-MI-08](#) `pendingFeePositiveImpliesCreditPositive`
- [VS-MI-09](#) `marketSettlementFeesBounded`
- [VS-MI-10](#) `marketContinuousFeeBounded`
- [VS-MI-11](#) `defaultSettlementFeesBounded`
- [VS-MI-12](#) `defaultContinuousFeeBounded`
- [VS-MI-13](#) `claimableAndWithdrawableBackedByBalance`
- [VS-MI-14](#) `collateralBackedByBalance`
- [VS-MI-15](#) `perTokenClaimableBounded`
- [VS-MI-16](#) `noSelfApprove`
- [VS-MI-17](#) `debtSumAndWithdrawableWithinTotalUnits`
- [VS-MI-18](#) `creditSumAndCfcEqualTotalUnitsWhenNoBadDebt`
- [VS-MI-20](#) `tickSpacingDividesDefault`

The many-market regime applies the same `requireInvariant` re-entry pattern, built in `setupValidStateManyMidnight`.

Unsafe Assumptions

These reduce verification scope to make the problem tractable for the prover. In the codebase, every `require` statement that constitutes an unsafe assumption is annotated with an `"UNSAFE: ..."` message string.

Three-User Narrowing ([midnight_one.spec](#), [midnight_many.spec](#)):

The `position[id][user]` mapping is narrowed to three symbolic users (`ghostMiPositionUserOne` / `Two` / `Three`) — the minimum set covering `take`'s two distinct positions (maker / taker) plus one bystander. `Sload/Sstore` hooks reject paths that touch a fourth user as infeasible. This is the same narrowing used in both regimes; conservation invariants (`Σ credit + cfc == totalUnits`, `Σ debt + withdrawable == totalUnits`, `Σ collateral[u][i] <= balance[token]`) take the sum over exactly these three users.

- `VALID_POSITION_USER(u)` is required on every position-field `Sload` and `Sstore`.
- Position fields are zero for every user outside the three-user set (one-mode and many-mode setup).
- Position `collateral[u][i]` is zero for every user outside the three-user set.

Two-Collateral / Five-Tick Models ([midnight.spec](#), [utils_lib.spec](#), [tick_lib.spec](#)):

`ghostNumCollaterals ∈ {1, 2}` narrows the per-market collateral count to a two-collateral model, just enough to cover `loanToken-vs-collateralToken` aliasing and pairs of distinct collaterals; `ghostNumTicks ∈ {1..5}` narrows the symbolic tick set to five points covering monotonicity boundary cases. The bitmap of activated collaterals is constrained to those slots accordingly.

- `market.collateralParams.length <= ghostNumCollaterals` (and equals `ghostNumCollaterals` once any market is touched).
- `VALID_COLLATERAL_BIT(i)` is required on every `position[u].collateral[i]` `Sload` / `Sstore`.
- `VALID_COLLATERAL_BITMAP(v)` is required on every `position[u].collateralBitmap` `Sload` / `Sstore`.
- Position `collateral[i]` is zero for every slot `i` outside the bitmap (one-mode and many-mode setup).

- `collateralLength` is synchronised with the count of non-zero collateral slots.
- `collateralLength <= ghostNumCollaterals`.

`touchMarket` Summary ([touch_market_summary.spec](#)):

The summary `touchMarketCVL` models only already-touched markets (the `if (tickSpacing == 0)` creation body is replaced with a `require(tickSpacing > 0)`-style precondition). Because `touchMarket` is public, the `internal` summary also intercepts the external parametric entry (the ABI wrapper calls the summarized internal implementation), so creation is not exercised in summarized confs. Creation is verified in the dedicated [market_creation.conf](#) (rules MC-MI-01..07: reachability satisfy, parameter validation, default-fee copy-down, tickSpacing init, idempotence, state frame, chainId/midnight identity); the expected-UNSAT diagnostic [confs/debug/touch_market_diag.conf](#) documents the interception in the summarized regime.

- `ghostMiMarketTickSpacing[id] > 0` is required to be true inside the summary.
- `market.loanToken` is required to be stable across all touches with the same id.
- `market.collateralParams[i].token` is required to be stable across all touches with the same id.

One-Market Regime Pins ([midnight.spec](#), [midnight_one.spec](#), [id_lib.spec](#)):

The one-market regime collapses the per-market surface to scalar ghosts. To keep that collapse sound, every code path that flows through `IdLib.toId` (e.g. `claimContinuousFee`, `updatePosition`, `toMarket`) must share the scalar `loanToken`; otherwise `claimContinuousFee` could drain `balance[loanToken]` for a token that does not match the scalar narrowing. The scalar narrowed market is assumed touched in the pre-state, and `claimableSettlementFee[t]` is constrained to be non-zero only for the scalar `loanToken`.

- `IdLib.toId` pins `market.loanToken == ghostMiOneMarketLoanToken` when the scalar is non-zero.
- `ghostMiOneMarketTickSpacing > 0` in the pre-state (scalar narrowed market is touched).
- `claimableSettlementFee[t] != 0 => t == ghostMiOneMarketLoanToken`.
- Scalar `collateralToken[0]` and `collateralToken[1]` are set by prior `touchMarket` and stable across calls; `collateralToken[1] != collateralToken[0]` (sort invariant restatement).

Many-Market Regime Narrowing ([midnight_many.spec](#), [midnight.spec](#)):

The many-market regime restricts the prover to three distinct touched markets (`ghostMiMarketIdA`, `ghostMiMarketIdB`, `ghostMiMarketIdC`, pairwise distinct) so the ERC-20-backing invariants can sum over a fixed market set. `VALID_MARKET_MANY(id)` is required on every per-id Sload / Sstore hook, and a matching initial-state require zeroes every per-id ghost outside the narrowing. A many-mode flag activates the no-aliasing requires in `touchMarketCVL` (`collateralParams[i].token != loanToken`).

- `VALID_MARKET_MANY(id)` is required on every per-id market / position Sload and Sstore.
- `tickSpacing[id] > 0 => VALID_MARKET_MANY(id)` (only `idA` / `idB` / `idC` may be touched).
- `tickSpacing[id] > 0 <=> ghostMiMarketLoanToken[id] != 0` (touched iff loan-token ghost set).
- Per-id market, position, and collateral-token ghosts are zero for every id outside the narrowing.
- `market.collateralParams[i].token != market.loanToken` is required when the many-mode flag is active.

Take-Run Tractability ([callbacks.spec](#), [ratifier.spec](#)):

`take` is the heaviest single-method run; to keep its SMT problem tractable the take-callback / sell-callback / repay-callback / liquidate-callback / flash-callback payloads are restricted to empty `data`, and the ratifier data is restricted to empty.

- `data.length == 0` on every callback payload.
- `data.length == 0` on `IRatifier.isRatified` ratifier data.

Bounded ERC-20 Account Set ([erc20.spec](#)):

ERC-20 balance and allowance lookups are restricted to a predefined account set (the same three position users plus a small bystander pool) so the symbolic balance ghost stays finite and the conservation laws stay decidable.

- ERC-20 `balanceOf`, `allowance`, `transfer*`, `approve` lookups are within the predefined account set.

Additional Setup Pins ([id_lib.spec](#), [midnight_one.spec](#), [midnight_many.spec](#)):

- One-mode: every touched market's `tickSpacing` equals the scalar mirror (the Sload hook anchors only the id-keyed ghost).
- Many-mode: `toId` pins `market.loanToken` / `collateralParams[0..1].token` to the id-attributed ghosts (justified by the real `idLib.toId` hashing the full struct).
- Many-mode: a collateral slot with unset token attribution carries no collateral pot (supplyCollateral always passes through touchMarket).

Prover Configuration:

- Loop unrolling is capped at **2 iterations** across all four conf files (matches `loop_iter: "2"`).
- Optimistic loop and optimistic hashing are enabled (`optimistic_loop: true`, `optimistic_hashing: true`, `hashing_length_bound: 1024`).
- The extended one-market config (`valid_state_one_ext.conf`) raises `smt_timeout` and fans the solver out across z3 / cvc5 / yices / bitwuzla with split-parallel mode; used as the retry config when the default solver budget is insufficient.

Trusted Assumptions

These assume that initialization has been completed and that admin-configured parameters fall within reasonable operational bounds. Since `Midnight`'s constructor body is not modelled in CVL (the constructor stores the initial `configurator`; its body is treated as opaque), the post-initialization state is taken as a precondition. In the codebase, every `require` statement that constitutes a trusted assumption is annotated with a `"TRUSTED: ..."` message string.

Role Setters (`midnight.spec`):

The role addresses are set by the constructor / `setConfigurator` / `setFeeSetter` / `setFeeClaimer` flows; the verification assumes they hold sane post-initialization values rather than re-verifying constructor wiring. `configurator` is asserted non-zero (the constructor sets it to `msg.sender`) and none of `configurator` / `feeSetter` / `feeClaimer` may equal the Midnight contract address itself.

- `ghostMiConfigurator != 0` (configurator set in constructor).
- `ghostMiConfigurator != _Midnight`.
- `ghostMiFeeSetter != _Midnight`.
- `ghostMiFeeClaimer != _Midnight`.

Market Maturity Horizon (`id_lib.spec`):

A created market (`tickSpacing > 0`) matures within 100 years of the current call — the direct consequence of `touchMarket`'s `MaturityTooFar` gate (`maturity <= block.timestamp + 100 * 365 days` at creation, src L758, proven enforced by MC-MI-02) plus time monotonicity. The inductive proofs cannot carry the creation-time history, so the gate is mirrored as a premise; it keeps `continuousFee * timeToMaturity < WAD (MAX_CONTINUOUS_FEE == floor(0.01e18 / 365 days)` — the two parameters are jointly tuned with no headroom beyond the floor rounding), the boundary beyond which `take`'s buyer leg would mint `pendingFee > credit` (VS-MI-01). Untouched ids carry no premise, so the creation gate itself stays falsifiable in `market_creation.conf`.

- one-mode: `ghostMiOneMarketTickSpacing == 0 || market.maturity <= e.block.timestamp + 3153600000`.
- many-mode: `ghostMiMarketTickSpacing[id] == 0 || market.maturity <= e.block.timestamp + 3153600000`.

Token Sanity (`midnight_one.spec`):

The scalar `loanToken` and `collateralToken[0]` are set by prior `touchMarket` calls; they must be non-zero (matches the protocol's sort-by-token invariant) and not the Midnight contract itself (the protocol does not borrow against its own address).

- `ghostMiOneMarketLoanToken != _Midnight` (loanToken is not the lending contract itself).
- `ghostMiOneCollateralToken[0] != _Midnight` (collateralToken[0] is not the lending contract itself).

Verification Properties

Links to specific CVL spec files are provided for each property, with status indicators in two columns:

- **Audit** -- status at the audit commit.
- **Mitig** -- status after the mitigation review (filled in once mitigations land and the prover is re-run). Empty cell on the first generation.

Status indicators:

-  Verified
-  Timeout (undecided within budget, no counterexample — see Verification Results)
-  Violated

Valid State

System-wide invariants that hold at every reachable state of `Midnight`. Each per-market invariant is proven for a single symbolic market under the one-market narrowing; because markets do not interfere, that proof generalises to every market — so the invariants are listed once below. The many-market regime additionally verifies the multi-market forms (VS-MI-19, the multi-market ERC-20 backing, and the cross-market rules HL-MI-22m / ST-MI-17) — expressed explicitly over the three narrowed markets (`idA`, `idB`, `idC`) — that a single-market narrowing cannot express.

Property	Name	Description	Audit	Mitig	Notes
VS-MI-01	<code>creditCoversPendingFee</code>	fee accrued on a lender position but not yet collected (pendingFee) never exceeds that lender's credit balance, so collecting the protocol's fee can never consume more units than the position actually holds <code>forall u. pendingFee[u] <= credit[u]</code>			timeout: take, withdraw
VS-MI-02	<code>positionLastLossFactorWithinMarket</code>	each position's recorded snapshot of the cumulative bad-debt socialization factor (lossFactor) never runs ahead of the market's current value, so the lazy slash applied to a lender on its next touch is always a well-defined, non-negative loss <code>forall u. lastLossFactor[u] <= lossFactor</code>			
VS-MI-03	<code>lastAccrualNotInFuture</code>	a position's last fee-accrual timestamp never lies in the future, so the continuous fee charged on borrower debt is always computed over a non-negative elapsed time <code>forall u. lastAccrual[u] <= block.timestamp</code>			
VS-MI-04	<code>collateralBitmapMatchesSlot</code>	the bitmap recording which collateral tokens a borrower has posted exactly mirrors the posted balances: a slot's bit is set if and only if the borrower holds a non-zero amount of that collateral, so health checks and liquidators never miss or double-count collateral <code>forall u, valid i. bit_i(collateralBitmap[u]) <=> collateral[u][i] > 0</code>			
VS-MI-05	<code>nonEmptyPositionImpliesTouched</code>	no lender credit, borrower debt, fee, or collateral record can exist in a market that was never created; an initialized market is recognizable by its non-zero tick spacing <code>forall u. (any position[u] field != 0) => tickSpacing > 0</code>			
VS-MI-06	<code>creditAndDebtMutuallyExclusive</code>	within a market a user is either a lender (holding interest-bearing credit units) or a borrower (owing debt), never both at once, so every position settles unambiguously on one side of the book <code>forall u. credit[u] == 0 OR debt[u] == 0</code>			
		any position carrying lender credit, a bad-debt			

VS-MI-07	<code>creditOrLastLossFactorImpliesLastAccrual</code>	socialization snapshot, or uncollected fee has been through fee accrual at least once — its accrual timestamp is set, so the lazy fee and loss bookkeeping always has a valid starting point <code>forall u. (credit[u] > 0 OR lastLossFactor[u] > 0 OR pendingFee[u] > 0) => lastAccrual[u] > 0</code>	✓		
VS-MI-08	<code>pendingFeePositiveImpliesCreditPositive</code>	uncollected fee (pendingFee) can only exist on a live lender position: once a lender's credit is fully gone, no pending fee remains owed on that position <code>forall u. pendingFee[u] > 0 => credit[u] > 0</code>	⌚		timeout: take
VS-MI-09	<code>marketSettlementFeesBounded</code>	each of the market's seven settlement-fee rates (the trading fee charged on take() fills, stored per time-to-maturity breakpoint in centi-basis-points) stays within its per-breakpoint protocol maximum, capping what a trade can ever be charged <code>forall i in 0..6. settlementFeeCbp_i <= MAX_SETTLEMENT_FEE_STORED_i()</code>	✓		
VS-MI-10	<code>marketContinuousFeeBounded</code>	the market's continuous fee rate — the ongoing fee accrued on borrower debt for the protocol — never exceeds the protocol-wide cap, bounding what borrowers can be charged <code>continuousFee <= MAX_CONTINUOUS_FEE_CVL()</code>	✓		
VS-MI-11	<code>defaultSettlementFeesBounded</code>	the per-loan-token default settlement-fee rates (the schedule copied into every newly created market) each stay within the same per-breakpoint protocol maximum as the live market fees, so no market can be born with an excessive trading fee <code>forall t, i in 0..6. defaultSettlementFeeCbp[t][i] <= MAX_SETTLEMENT_FEE_STORED_i()</code>	✓		
VS-MI-12	<code>defaultContinuousFeeBounded</code>	the per-loan-token default continuous fee (copied into every newly created market) never exceeds the protocol-wide cap, so no market can be born charging borrowers above it <code>forall t. defaultContinuousFee[t] <= MAX_CONTINUOUS_FEE_CVL()</code>	✓		
VS-MI-13	<code>claimableAndWithdrawableBackedByBalance</code>	the protocol's loan-token balance always covers everything payable on demand in that token: the settlement-fee pot claimable by the fee claimer, the loan tokens currently available for withdrawal from the market (withdrawable), and any posted collateral that happens to be denominated in the loan token itself <code>balance[loanToken][Midnight] >= claimableSettlementFee[loanToken] + withdrawable + Σ_3users collateral_in_loanToken</code>	✓		
VS-MI-14	<code>collateralBackedByBalance</code>	the protocol's balance of each collateral token covers the total collateral posted by borrowers in that token (summed over the three modeled users), so every borrower's collateral can always be returned or seized in full <code>forall valid i. balance[collateralToken[i]][Midnight] >= Σ_3users collateral[u][i]</code>	✓		
VS-MI-15	<code>perTokenClaimableBounded</code>	for every token, the protocol's balance covers the settlement-fee pot owed to the fee claimer in that token, so a fee claim can always be paid out <code>forall t. balance[t][Midnight] >= claimableSettlementFee[t]</code>	✓		
VS-MI-16	<code>noSelfApprove</code>	the protocol never grants an ERC20 spending allowance to itself on any token, so no code path can move the protocol's own funds through a self-directed transferFrom <code>forall t. allowance[t][Midnight][Midnight] == 0</code>	✓		

VS-MI-17	<code>debtSumAndWithdrawableWithinTotalUnits</code>	loan-side conservation: every unit of the market's total loan units (totalUnits) is either lent out as some borrower's debt or sitting as loan tokens currently available for withdrawal from the market (withdrawable) — debt summed over the three modeled users <pre>$\sum_3users\ debt[u] + withdrawable == totalUnits$</pre>	✓		
VS-MI-18	<code>creditSumAndCfcEqualTotalUnitsWhenNoBadDebt</code>	while no bad debt has been socialized, the market's total loan units (totalUnits) are fully and exactly attributed to lender credit plus the continuous-fee credit (cfc) — the fee units accrued to the protocol and claimable by the fee claimer — with credit summed over the three modeled users <pre>$lossFactor == 0 \Rightarrow \sum_3users\ credit[u] + continuousFeeCredit == totalUnits$</pre>	🕒		timeout: take
VS-MI-19	<code>untouchedMarketIsEmptyParametric</code>	a market that was never created holds nothing: while its offer-price tick spacing is unset, all of its accounting — total loan units, bad-debt socialization factor, withdrawable liquidity, fee credit, and every fee parameter — is zero <pre>$\text{forall } id.\ tickSpacing[id] == 0 \Rightarrow (\text{all other } MarketState[id]\ fields == 0)$</pre>	✓		
VS-MI-20	<code>tickSpacingDividesDefault</code>	every market's tick spacing — the granularity of offer price ticks in the take() trade entry point — is a divisor of the protocol default: it starts at 4 on market creation and can only ever be refined to 2 or 1, never coarsened (0 marks an uncreated market) <pre>$\text{forall } id.\ tickSpacing[id] \in \{0, 1, 2, 4\}$</pre>	✓		
VS-MI-21	<code>debtPositiveImpliesCollateralBitmapNonZero</code>	a borrower carrying live debt always has at least one collateral slot on record, so outstanding debt is never left with nothing for a liquidator to seize — whenever the last collateral is taken, the debt is either cleared exactly or realized as bad debt and the position zeroed <pre>$\text{forall } u.\ debt[u] > 0 \Rightarrow collateralBitmap[u] != 0$</pre>	🕒		timeout: liquidate
VS-MI-22	<code>continuousFeeCreditWithinTotalUnitsMinusDebt</code>	the continuous-fee credit (cfc) — the fee units accrued to the protocol and claimable by the fee claimer — plus all outstanding borrower debt fits within the market's total loan units (totalUnits); equivalently the fee pot never exceeds the withdrawable loan tokens, so claiming the continuous fee can always be paid out (debt summed over the three modeled users) <pre>$continuousFeeCredit + \sum_3users\ debt[u] \leq totalUnits$</pre>	✗		Issue #1 (INFO)

High-Level

End-to-end behavioral guarantees on `Midnight` entry points. Verified against the one-market regime (the full valid-state invariant set is loaded as preconditions via `setupValidStateOneMidnight`); no high-level rule needs cross-id reasoning, so all are checked under the single-market narrowing.

Property	Name	Description	Audit	Mitig	Notes
HL-MI-01	<code>claimableSettlementFeeNeverDecreases</code>	settlement fees owed to the fee claimer are never clawed back by trading — the take() trade entry point (a buyer fills a maker's offer) can only grow, never shrink, the pot of claimable settlement fees for any token <pre>$\text{forall } token.\ claimableSettlementFee[token] \geq claimableSettlementFee[token]$</pre>	✓		
		in the take() trade entry point the buyer pays at least as much as the seller receives, and that spread is the settlement fee: the growth of the fee pot claimable by the fee claimer must exactly equal the loan tokens that flow into the protocol's own balance during the trade — the protocol neither skims tokens beyond the			

HL-MI-02	<code>takeCapturesExactSettlementFee</code>	recorded fee nor records fees it never received (checked with no trade participant, callback, or payout receiver being the protocol itself) <pre>claimableSettlementFee[loanToken]' - claimableSettlementFee[loanToken] == balance[loanToken][Midnight]' - balance[loanToken][Midnight]</pre>	✓		
HL-MI-03	<code>withdrawExactDecrement</code>	withdraw removes exactly the requested amount from the market's available liquidity and total units and sends exactly that amount of loan tokens out to the receiver; the lender's position legs are exact too — credit drops by exactly the withdrawn units (measured after any pending bad-debt slash and fee accrual settle), pendingFee burns proportionally to the withdrawn credit rounded against the lender, and a full exit leaves no pendingFee behind, so a lender cannot time their exit to dodge a pending slash <pre>withdrawable' == withdrawable - units AND totalUnits' == totalUnits - units AND balance[loanToken][Midnight]' == balance[loanToken][Midnight] - units AND credit[onBehalf]' == viewCredit - units AND viewCredit > 0 => pendingFee[onBehalf]' == viewPendingFee - ceil(viewPendingFee * units / viewCredit) AND units == viewCredit => pendingFee[onBehalf]' == 0 where (viewCredit, viewPendingFee) = the position after its pending slash/accrual settles</pre>	🕒		timeout: withdraw
HL-MI-04	<code>repayExactSwap</code>	repaying converts a borrower's debt back into available liquidity one-for-one — debt drops by exactly the repaid units, the withdrawable liquidity rises by the same amount, the market's total loan units are unchanged, and exactly that many loan tokens are pulled into the protocol <pre>debt[onBehalf]' == debt[onBehalf] - units AND withdrawable' == withdrawable + units AND totalUnits' == totalUnits AND balance[loanToken][Midnight]' == balance[loanToken][Midnight] + units</pre>	✓		
HL-MI-05	<code>claimContinuousFeeExactDecrement</code>	when the fee claimer collects the protocol's continuous-fee credit (cfc — fee units accrued to the protocol), the claim is exact: the cfc pot, the market's total loan units, and the withdrawable liquidity each drop by exactly the claimed amount, and exactly that many loan tokens leave the protocol <pre>continuousFeeCredit' == continuousFeeCredit - amount AND totalUnits' == totalUnits - amount AND withdrawable' == withdrawable - amount AND balance[loanToken][Midnight]' == balance[loanToken][Midnight] - amount</pre>	✓		
HL-MI-06	<code>supplyCollateralExactAdd</code>	supplying collateral credits the borrower's collateral slot by exactly the deposited amount and pulls exactly that many collateral tokens into the protocol — no fees, rounding, or leakage on the way in <pre>collateral[onBehalf][i]' == collateral[onBehalf][i] + assets AND balance[collateralToken[i]][Midnight]' == balance[collateralToken[i]][Midnight] + assets</pre>	✓		
		flash loans are free and side-effect free — the protocol lends tokens out and pulls the same amount back within the call, charging no fee, so neither the protocol nor the caller profits, and a flash loan cannot be used to move any lender or borrower position, market aggregate, fee pot, offer-fill counter, or delegation <pre>forall token, u, i in {0, 1}, g, a, b. balance[token][Midnight]' == balance[token] [Midnight] AND credit[u]' == credit[u] AND debt[u]' == debt[u] AND pendingFee[u]' ==</pre>			

HL-MI-07	<code>flashLoanBalanceNeutral</code>	<code>pendingFee[u] AND lastLossFactor[u]' == lastLossFactor[u] AND lastAccrual[u]' == lastAccrual[u] AND collateralBitmap[u]' == collateralBitmap[u] AND collateral[u][i]' == collateral[u][i] AND totalUnits' == totalUnits AND withdrawable' == withdrawable AND continuousFeeCredit' == continuousFeeCredit AND lossFactor' == lossFactor AND claimableSettlementFee[token]' == claimableSettlementFee[token] AND consumed[u][g]' == consumed[u][g] AND isAuthorized[a][b]' == isAuthorized[a][b]</code>	✓		
HL-MI-08	<code>withdrawCollateralLeavesBorrowerHealthy</code>	a borrower can only take collateral out if their position ends up healthy — whenever <code>withdrawCollateral</code> succeeds, the borrower's remaining collateral still covers their debt under the loan-to-liquidation-value threshold (ltlv), so withdrawing collateral can never leave the position liquidatable <code>withdrawCollateral succeeds => isHealthy(market, onBehalf)'</code>	✓		
HL-MI-09	<code>liquidateIsReductive</code>	liquidation only shrinks the borrower's position, and by exact amounts: the targeted collateral slot drops by exactly the assets the liquidator seizes, no other collateral slot is touched, and the debt reduction splits exactly into the cash the liquidator repaid plus the bad debt written off and socialized across lenders (mirrored one-for-one as the only drop in the market's total loan units) <code>collateral[borrower][i] - collateral[borrower][i]' == seized AND collateral[borrower][other]' == collateral[borrower][other] AND debt[borrower] - debt[borrower]' == repaid + (totalUnits - totalUnits')</code>	✓		
HL-MI-10	<code>collateralRoundTripRestoresSlot</code>	collateral is not fee-bearing — depositing collateral and then withdrawing the same amount from the same slot restores the borrower's collateral balance exactly, with no leakage in either direction <code>after supplyCollateral(i, assets); withdrawCollateral(i, assets): collateral[onBehalf][i]' == collateral[onBehalf][i]</code>	✓		
HL-MI-11	<code>updatePositionPreservesTotalUnitsAndWithdrawable</code>	settling a lender's position (applying the continuous-fee accrual and any pending bad-debt slash) only reshuffles value between the lender's credit, their fee accrued but not yet collected (pendingFee), and the protocol's fee pot — it never changes the market's total loan units or the withdrawable liquidity <code>totalUnits' == totalUnits AND withdrawable' == withdrawable</code>	✓		

The next eleven (HL-MI-12..22) are **bug-hunting / offensive** rules — they assert protocol-safety properties whose refutation would reveal a real bug (leak, insolvency, over-seizure, missing auth, mis-accounting, view/state divergence). Unlike HL-MI-01..11 they are expected to possibly fail; a failure is a finding to investigate.

Property	Name	Description	Audit	Mitig	Notes
HL-MI-12	<code>loanTokenSurplusNonDecreasing</code>	the protocol never pays out more loan tokens than the liability it discharges — its loan-token surplus (token balance minus the settlement fees owed to the fee claimer, minus the withdrawable liquidity, minus any borrower collateral denominated in the loan token) never decreases under any operation; a decrease would be a token leak <code>forall f. surplus' >= surplus where surplus = balance[loanToken][Midnight] -</code>	✓		

		<code>claimableSettlementFee[loanToken] - withdrawable - $\sum_{u,i} collateral[u][i]$ over slots whose token == loanToken</code>			
HL-MI-13	<code>collateralTokenSurplusNonDecreasing</code>	the protocol always holds at least as many collateral tokens as it owes back to borrowers — for a live collateral slot whose token differs from the loan token, the surplus (the protocol's token balance minus the collateral recorded for the three modeled users) never decreases under any operation; a decrease would be a collateral leak <code>forall f. surplus' >= surplus where surplus = balance[collateralToken[i]][Midnight] - $\sum_u$ collateral[u][i]</code>	✓		
HL-MI-14	<code>updatePositionViewMatchesState</code>	the read-only position preview tells the truth — the credit, pending fee, and accrued fee that <code>updatePositionView</code> predicts exactly match what a real settlement (<code>updatePosition</code>) writes to storage and returns; the continuous-fee credit grows by exactly the previewed accrued fee (no fee-pot minting); and the position's bookkeeping stamps are refreshed to the current time and the market's current lossFactor <code>credit[u]' == viewCredit AND pendingFee[u]' == viewPendingFee AND continuousFeeCredit' == continuousFeeCredit + viewAccrued AND lastAccrual[u]' == block.timestamp AND lastLossFactor[u]' == lossFactor AND updatePosition returns (viewCredit, viewPendingFee, viewAccrued)</code>	✓		
HL-MI-15	<code>liquidateRespectsLifSeizureBound</code>	a liquidator's bonus is capped — the oracle value of the collateral seized never exceeds the debt repaid times the liquidation incentive factor (lif), the WAD-scaled bonus multiplier that ramps from 1.0 at maturity up to <code>maxLif</code> over 60 minutes; in post-maturity mode the bound uses the actual time-ramped lif (so a liquidator just past maturity cannot collect the full <code>maxLif</code> bonus), and the value seized never exceeds <code>repaid * maxLif</code> in any mode <code>seized * price * WAD <= repaid * lif * ORACLE_PRICE_SCALE AND seized * price * WAD <= repaid * maxLif * ORACLE_PRICE_SCALE where lif = postMaturityMode ? min(maxLif, WAD + floor((maxLif - WAD) * max(0, now - maturity) / 3600)) : maxLif</code>	🕒		timeout: liquidate
HL-MI-16	<code>withdrawRequiresAuthorization</code>	only the position owner, or someone the owner delegated to before the call, can withdraw a lender's funds — any successful withdraw implies the caller already was the <code>onBehalf</code> account or held its authorization at entry <code>withdraw(onBehalf) succeeds => e.msg.sender == onBehalf OR isAuthorized[onBehalf] [e.msg.sender] (pre-state)</code>	✓		
HL-MI-17	<code>withdrawCollateralRequiresAuthorization</code>	only the position owner, or someone the owner delegated to before the call, can pull a borrower's collateral — any successful <code>withdrawCollateral</code> implies the caller already was the <code>onBehalf</code> account or held its authorization at entry <code>withdrawCollateral(onBehalf) succeeds => e.msg.sender == onBehalf OR isAuthorized[onBehalf][e.msg.sender] (pre- state)</code>	✓		
		socialized losses are never quietly reversed — the			

HL-MI-18	<code>lossFactorMonotonic</code>	cumulative bad-debt socialization factor (lossFactor), which determines how much each lender position is slashed on its next touch, never decreases under any operation <pre>forall f. lossFactor' >= lossFactor</pre>	✓		
HL-MI-19	<code>slashNeverMintsCredit</code>	the lazy settlement of a lender position (the pending bad-debt slash plus fee accrual) can only take value from the lender, never create it — the credit a settlement would realize never exceeds the credit currently stored for the position <pre>viewCredit <= credit[u] where viewCredit = the credit the position's pending slash/accrual would realize</pre>	✓		
HL-MI-20	<code>takeDoesNotTouchBystander</code>	a trade settles only between its two counterparties — the take() trade entry point leaves every stored field of any user who is neither the maker nor the taker untouched, including the bookkeeping stamps, the collateral bitmap, and that user's offer-fill counter <pre>forall bystander not in {maker, taker}, i in {0, 1}. credit[bystander]' == credit[bystander] AND debt[bystander]' == debt[bystander] AND pendingFee[bystander]' == pendingFee[bystander] AND lastLossFactor[bystander]' == lastLossFactor[bystander] AND lastAccrual[bystander]' == lastAccrual[bystander] AND collateralBitmap[bystander]' == collateralBitmap[bystander] AND collateral[bystander][i]' == collateral[bystander][i] AND consumed[bystander][offer.group]' == consumed[bystander][offer.group]</pre>	✓		
HL-MI-21	<code>consumedBoundedByOfferMax</code>	an offer can never be over-filled — the maker's cumulative fill counter (consumed) stays within the offer's cap, and advances by exactly the size of this fill: the capped-side assets for an assets-capped offer, the filled units for a units-capped one; a dropped or under-counted increment, which would allow unbounded aggregate over-fill or replay, refutes the exact-increment legs <pre>(offer.maxAssets > 0 => consumed[maker][group]' <= offer.maxAssets) AND (offer.maxAssets == 0 => consumed[maker][group]' <= offer.maxUnits) AND (offer.maxAssets > 0 => consumed[maker][group]' == consumed[maker][group] + (offer.buy ? buyerAssets : sellerAssets)) AND (offer.maxAssets == 0 => consumed[maker][group]' == consumed[maker][group] + units)</pre>	✓		
HL-MI-22	<code>gettersMatchStorage</code>	the public view getters report the truth — credit, debt, totalUnits, and withdrawable each return exactly the underlying stored value, so off-chain integrations and on-chain callers see the real position and market state <pre>credit(id, u) == credit[u] AND debt(id, u) == debt[u] AND totalUnits(id) == totalUnits AND withdrawable(id) == withdrawable</pre>	✓		

HL-MI-23..62 cover complex financial scenarios: exact loss-socialization and fee equations, post-slash solvency, take pricing and fill-accounting exactness, liquidation bounds, and counterparty token routing. The light tier runs in [high_level.conf](#); the heavy nonlinear tier (slash/lossFactor equations, take pricing exactness, inductive solvency) runs in [high_level_heavy.conf](#) with the extended smt budget.

Property	Name	Description	Audit	Mitig	Notes
HL-MI-23	<code>accrualConservesCreditIntoFeePot</code>	settling a lender position conserves value into the fee pot — the continuous-fee credit grows by exactly the accrued fee the settlement reports, and the pot never gains more than the lender's credit loses; with no pending bad-debt slash the conservation is exact three ways, leaving the position's face value (credit - pendingFee) unchanged <code>continuousFeeCredit' == continuousFeeCredit + accruedFee AND credit[u] - credit[u]' >= accruedFee AND (lastLossFactor[u] == lossFactor AND lossFactor < max_uint128) => (credit[u] - credit[u]' == accruedFee AND pendingFee[u] - pendingFee[u]' == accruedFee AND credit[u] - pendingFee[u] == credit[u]' - pendingFee[u]')</code>			
HL-MI-24	<code>takeBuyerFeePreChargeExactAndBounded</code>	when a buyer gains credit through the take() trade entry point, the up-front fee charge is exactly linear in the minted credit and the time to maturity, and never exceeds the credit minted — so neither fee evasion by splitting a take into pieces nor inflation of the pre-charge is possible (deltas measured against the position after its pending slash/accrual settles) <code>pendingFee[buyer]' - viewPendingFee == floor((credit[buyer]' - viewCredit) * continuousFee * ttm / WAD) AND pendingFee[buyer]' - viewPendingFee <= credit[buyer]' - viewCredit where ttm = max(0, maturity - now)</code>			timeout: take
HL-MI-25	<code>takeSellerBurnsPendingFeeProportionally</code>	when a seller's credit is consumed through the take() trade entry point, the seller's fee accrued but not yet collected (pendingFee) burns exactly in proportion to the credit consumed, rounded up against the seller — selling credit cannot shed a smaller share of the pending fee than of the credit <code>viewCredit > 0 => pendingFee[seller]' == viewPendingFee - ceil(viewPendingFee * (viewCredit - credit[seller]') / viewCredit) where (viewCredit, viewPendingFee) = the seller's position after its pending slash/accrual settles</code>			timeout: take
HL-MI-26	<code>updatePositionIdempotentSameBlock</code>	settling the same lender position twice at one timestamp charges nothing twice — the second updatePosition in the same block reports zero accrued fee and leaves the lender's credit, pending fee, and the continuous-fee credit pot exactly where the first call put them: no double accrual, no double slash, no fee-pot drift <code>after updatePosition(u), a second updatePosition(u) at the same timestamp satisfies accruedFee == 0 AND credit[u]' == credit[u] AND pendingFee[u]' == pendingFee[u] AND continuousFeeCredit' == continuousFeeCredit</code>			
HL-MI-27	<code>accrualLinearInTimeWithMaturityCutoff</code>	with no pending bad-debt slash, the continuous fee accrues exactly linearly in time — a settlement collects the lender's pendingFee scaled by the fraction of the time between the last settlement and maturity that has since elapsed; nothing accrues once the position is stamped at or after maturity, and a settlement at or after maturity collects the entire remaining pendingFee <code>lastAccrual[u] < maturity => accruedFee == floor(pendingFee[u] * (min(now, maturity)</code>			

		<pre>- lastAccrual[u] / (maturity - lastAccrual[u])) AND lastAccrual[u] >= maturity => accruedFee == 0 AND (now >= maturity AND lastAccrual[u] < maturity) => pendingFee[u]' == 0</pre>			
HL-MI-28	feeAccrualMonotoneAndFrozenAfterMaturity	viewed at two moments over the same stored state (t1 <= t2), a lender position's fee surface only moves one way — the accrued fee grows with time and the realizable credit never grows by waiting; once maturity has passed the surface is frozen entirely, and by maturity the full pendingFee has converted into collectable fee <pre>t1 <= t2 => (accruedFee(t2) >= accruedFee(t1) AND viewCredit(t2) <= viewCredit(t1) AND (t1 >= maturity => (viewCredit, viewPendingFee, accruedFee)(t1) == (viewCredit, viewPendingFee, accruedFee)(t2)) AND (t2 >= maturity AND lastAccrual[u] < maturity => viewPendingFee(t2) == 0))</pre>	✓		
HL-MI-29	liquidateOnlyWhenUnhealthyOrPastMaturity	a borrower who is still solvent can never be liquidated — a normal-mode liquidation only succeeds against a borrower whose collateral no longer covered their debt under the ltv at entry, and the post-maturity liquidation mode is only available strictly after the market's maturity <pre>liquidate(postMaturityMode == false) succeeds => NOT isHealthy(market, borrower) (pre-state) AND liquidate(postMaturityMode == true) succeeds => block.timestamp > maturity</pre>	✓		
HL-MI-30	lossFactorUpdateExact	lenders are slashed by exactly the share of value that bad debt destroys — when a liquidation writes off bad debt, the lossFactor advances by precisely the formula matching the fraction of the market's total loan units wiped out, and it does not move at all when no bad debt is realized <pre>totalUnits' == totalUnits => lossFactor' == lossFactor AND totalUnits' < totalUnits => lossFactor' == max_uint128 - floor((max_uint128 - lossFactor) * totalUnits' / totalUnits)</pre>	✓		
HL-MI-31	cfcRescaleExact	the protocol shares every socialized loss with the lenders — on a bad-debt write-off, the continuous-fee credit is haircut by exactly the same slash factor lender positions bear, so the fee claimer can neither dodge nor over-pay the loss; with no bad debt the pot is untouched by liquidate <pre>totalUnits' == totalUnits => continuousFeeCredit' == continuousFeeCredit AND (totalUnits' < totalUnits AND lossFactor < max_uint128) => continuousFeeCredit' == floor(continuousFeeCredit * (max_uint128 - lossFactor') / (max_uint128 - lossFactor)) AND (totalUnits' < totalUnits AND lossFactor == max_uint128) => continuousFeeCredit' == 0</pre>	✓		
HL-MI-32	liquidateLoanInCollateralOutExact	liquidation's money legs are exact on the protocol's books — the loan tokens the liquidator repays land one-for-one in the withdrawable liquidity and in the protocol's loan-token balance, while exactly the seized amount of collateral leaves both the protocol's collateral-token balance and the borrower's recorded slot (checked with the collateral token distinct from the loan token and neither the receiver nor the payer being the protocol) <pre>withdrawable' == withdrawable + repaid AND balance[loanToken][Midnight]' == balance[loanToken][Midnight] + repaid AND balance[collateralToken[i]][Midnight]' == balance[collateralToken[i]][Midnight] - seized AND collateral[borrower][i]' == collateral[borrower][i] - seized where</pre>	✓		

		<code>(seized, repaid) = liquidate's returned amounts</code>			
HL-MI-33	<code>lossFactorRiseImpliesUndercollateralizedAtMaxLif</code>	lenders are only ever forced to absorb bad debt from a genuinely insolvent borrower — the lossFactor can only rise when even seizing all of the borrower's collateral at the maximum liquidation bonus would not cover their debt (checked in a single-collateral configuration) <pre>lossFactor' > lossFactor => ceil(ceil(collateral[borrower][0] * price / ORACLE_PRICE_SCALE) * WAD / maxLif) < debt[borrower] (pre-state)</pre>	✓		
HL-MI-34	<code>liquidateDoesNotTouchBystander</code>	liquidating one borrower never touches anyone else's stored position — other lenders and borrowers absorb the socialized loss only lazily, through the market-level lossFactor, when their own position is next settled <pre>forall bystander != borrower. credit[bystander]' == credit[bystander] AND debt[bystander]' == debt[bystander] AND pendingFee[bystander]' == pendingFee[bystander] AND lastAccrual[bystander]' == lastAccrual[bystander] AND lastLossFactor[bystander]' == lastLossFactor[bystander] AND collateralBitmap[bystander]' == collateralBitmap[bystander] AND collateral[bystander][i]' == collateral[bystander][i]</pre>	✓		
HL-MI-35	<code>rcfDustEscapeRequiresDustCollateral</code>	a normal-mode liquidation is capped — the liquidator may not repay more than what restores the borrower to health (the recovery close factor, RCF) — except through the documented dust escape: whenever the repayment exceeds the cap, the collateral value that would remain above the cap must be below the market's dust threshold, so a liquidator can never over-liquidate a borrower with non-dust collateral left (repaid-input path, single collateral, ltv < WAD) <pre>repaid > maxRepaid => max(0, dustValue - maxRepaid) < rcfThreshold where collValue = floor(collateral[borrower][0] * price / ORACLE_PRICE_SCALE); maxDebt = floor(collValue * ltv / WAD); dustValue = floor(collValue * WAD / maxLif); maxRepaid = ceil((debt[borrower] - maxDebt) * WAD^2 / (WAD^2 - maxLif * ltv)) (pre-state)</pre>	✓		
HL-MI-36	<code>lossFactorMaxOnlyWhenUnitsWiped</code>	a market is only ever "bricked" — the lossFactor saturated at its maximum, after which trading halts and lender value is fully wiped — when the loss genuinely consumed the market: wiping all of the market's total loan units forces the brick, and a fresh brick can only occur when the units that survive the write-off are below the slash's rounding dust <pre>totalUnits' == 0 => lossFactor' == max_uint128 AND (lossFactor' == max_uint128 AND lossFactor < max_uint128) => (max_uint128 - lossFactor) * totalUnits' < totalUnits</pre>	✓		
HL-MI-37	<code>postSlashSolvencyOneStep</code>	the market stays solvent even with socialized losses still unrealized — the credit every lender could still claim after their pending bad-debt slash, summed over the three modeled users, plus the continuous-fee credit, never grows past the market's total loan units by more than one indivisible unit of rounding when the bound held before the operation <pre>forall f. realizableValue <= totalUnits => realizableValue' <= totalUnits' + 1 where realizableValue = Σ_u postSlashCredit(u) + continuousFeeCredit; postSlashCredit(u) = lastLossFactor[u] < max_uint128 ? floor(credit[u] * (max_uint128 - lossFactor) / (max_uint128 -</pre>	🕒		timeout: take, withdraw

		<code>lastLossFactor[u])) : 0</code>			
HL-MI-38	<code>slashBurnsPendingFeeProportionally</code>	a bad-debt slash never shifts the burden toward the fee — settling a lender's position (applying the pending slash and fee accrual) never increases the ratio of pendingFee to credit, so a slashed lender is never left owing proportionally more fee on less credit <pre>viewPendingFee * credit[u] <= pendingFee[u] * viewCredit where (viewCredit, viewPendingFee) = the position after its pending slash/accrual settles</pre>	✓		
HL-MI-39	<code>idleLenderCreditNonIncreasing</code>	a sleeping lender can never profit from someone else's loss event — across any operation that leaves the lender's own stored position untouched, the credit and the pendingFee that the lender would realize on their next settlement never increase <pre>forall f. f leaves u's stored position untouched => viewCredit' <= viewCredit AND viewPendingFee' <= viewPendingFee where (viewCredit, viewPendingFee) = the position after its pending slash/accrual settles</pre>	⌚		timeout: liquidate
HL-MI-40	<code>slashTimingFairness</code>	a lender cannot dodge socialized losses by timing their interactions — starting from identical positions, a lender who settles between two bad-debt events (taking two compounded rounded slashes) never ends up with more credit than an identical lender who sleeps through both and is slashed once <pre>credit[A] == credit[B] AND pendingFee[A] == pendingFee[B] AND lastLossFactor[A] == lastLossFactor[B] AND lastAccrual[A] == lastAccrual[B] => after liquidate; updatePosition(A); liquidate; updatePosition(B); updatePosition(A): credit[A]' <= credit[B]'</pre>	⌚		timeout: updatePosition, liquidate
HL-MI-41	<code>takeNettingUnitConservation</code>	the matching engine neither mints nor destroys loan units — in the take() trade entry point, the buyer's net position (credit minus debt) rises by exactly the filled units and the seller's falls by exactly the same, each measured against the position after its pending bad-debt slash and fee accrual settle <pre>(credit[buyer]' - viewCredit_buyer) + (debt[buyer] - debt[buyer]') == units AND (viewCredit_seller - credit[seller]') + (debt[seller]' - debt[seller]) == units</pre>	✓		
HL-MI-42	<code>takeFeeIncidenceMatchesLeviedFee</code>	the settlement fee the protocol captures on a trade matches its published fee schedule — the growth of the claimable settlement-fee pot equals the filled units times the market's time-interpolated settlement-fee rate, to within the tight rounding of the two pricing legs <pre>levied - 1 <= claimableSettlementFee[loanToken]' - claimableSettlementFee[loanToken] <= levied + 2 where levied = floor(units * settlementFee(id, max(0, maturity - now)) / WAD)</pre>	⌚		timeout: take
HL-MI-43	<code>reduceOnlyHonoredForMaker</code>	a maker who flags an offer reduce-only can only have their exposure shrunk by fills — a reduce-only buy offer never grows the maker's credit and a reduce-only sell offer never grows the maker's debt, so a fill cannot push the maker into a larger position than they signed for <pre>offer.reduceOnly AND offer.buy => credit[maker]' <= credit[maker] AND offer.reduceOnly AND NOT offer.buy => debt[maker]' <= debt[maker]</pre>	✓		
		a fill only succeeds when the offer's integrity gates all held at entry — the maker had authorized the offer's ratifier (the maker's protection against forged offers), the offer's time window was open, the offer's price tick sat on the market's tick grid, and the market was not			

HL-MI-44	<code>takeHonorsOfferIntegrityGates</code>	bricked by a saturated lossFactor <pre>take succeeds => isAuthorized[offer.maker] [offer.ratifier] (pre-state) AND offer.start <= block.timestamp <= offer.expiry AND tickSpacing > 0 AND offer.tick % tickSpacing == 0 AND lossFactor < max_uint128</pre>	✓		
HL-MI-45	<code>takeFillAccountingExact</code>	a fill is priced exactly at the offer's tick with maker-favoring rounding — for a buy offer both money legs round down (the maker-buyer pays the floor price, the seller receives the floor of price minus fee), for a sell offer both round up (the maker-seller receives the ceiling price, the buyer pays the ceiling of price plus fee) — and the maker's cumulative fill counter advances by exactly the capped side of this fill <pre>offer.buy => buyerAssets == floor(units * p / WAD) AND sellerAssets == floor(units * (p - sf) / WAD) AND NOT offer.buy => sellerAssets == ceil(units * p / WAD) AND buyerAssets == ceil(units * (p + sf) / WAD) AND consumed[maker][group]' == consumed[maker][group] + (offer.maxAssets > 0 ? (offer.buy ? buyerAssets : sellerAssets) : units) where p = tickPrice(offer.tick); sf = settlementFee(id, max(0, maturity - now))</pre>	✓		
HL-MI-46	<code>takeSettlementSpreadCappedByProtocolMax</code>	a trade can never be charged more settlement fee than the protocol's hard cap — the spread captured into the claimable settlement-fee pot stays within the filled units times the maximum settlement-fee rate (0.5% of the 1e18 fixed-point scale, WAD), plus one wei of rounding <pre>(claimableSettlementFee[loanToken]' - claimableSettlementFee[loanToken]) * WAD <= units * MAX_SETTLEMENT_FEE_360_DAYS + WAD where MAX_SETTLEMENT_FEE_360_DAYS == 0.005e18</pre>	🕒		timeout: take
HL-MI-47	<code>takeSellRoutesPayerReceiverMidnightExactly</code>	when a maker's sell offer is filled, the loan tokens flow between exactly the right wallets — the buyer-side payer (the caller, when no taker callback is set) pays exactly the buyer's price, the receiver designated in the maker's signed offer gets exactly the seller's proceeds, the protocol keeps exactly the fee spread, and no other wallet's loan-token balance moves <pre>buyerAssets >= sellerAssets AND balance[loanToken][payer]' == balance[loanToken][payer] - buyerAssets AND balance[loanToken][receiver]' == balance[loanToken][receiver] + sellerAssets AND balance[loanToken] [Midnight]' == balance[loanToken] [Midnight] + buyerAssets - sellerAssets AND balance[loanToken][bystander]' == balance[loanToken][bystander] where payer = msg.sender; receiver = offer.receiverIfMakerIsSeller</pre>	✓		
HL-MI-48	<code>takeBuyRoutesPayerReceiverMidnightExactly</code>	when a maker's buy offer is filled, the loan tokens flow between exactly the right wallets — the maker (the buyer, when no maker callback is set) pays exactly the buyer's price, the taker-chosen receiver gets exactly the seller's proceeds, the protocol keeps exactly the fee spread, and no other wallet's loan-token balance moves <pre>buyerAssets >= sellerAssets AND balance[loanToken][payer]' == balance[loanToken][payer] - buyerAssets AND balance[loanToken][receiver]' == balance[loanToken][receiver] + sellerAssets AND balance[loanToken] [Midnight]' == balance[loanToken] [Midnight] + buyerAssets - sellerAssets AND balance[loanToken][bystander]' == balance[loanToken][bystander] where payer = offer.maker; receiver =</pre>	✓		

		receiverIfTakerIsSeller			
HL-MI-49	consumedMonotoneGlobally	offer fills are irreversible — a maker's cumulative fill counter for any offer group can never be rolled back by any entry point, so a filled offer cap cannot be quietly reopened for replay forall f, user, group. consumed[user][group] ' >= consumed[user][group]	✓		
HL-MI-50	isHealthyMatchesFormula	the health check that gates collateral withdrawals and liquidations computes exactly the documented formula — a borrower is healthy precisely when they have no debt, or the sum over their collateral slots of the oracle value discounted by each slot's ltv covers the debt, with every term rounded down against the borrower (checked over the two modeled collateral slots) isHealthy(market, u) <=> (debt[u] == 0 OR $\sum_{i \text{ in } \{0,1\}} \text{floor}(\text{floor}(\text{collateral}[u][i] * \text{price}_i / \text{ORACLE_PRICE_SCALE}) * \text{ltv}_i / \text{WAD}) \geq \text{debt}[u])$	✓		
HL-MI-51	repayPullsExactlyFromPayerOnly	repaying a debt pulls the loan tokens from the caller's wallet only — exactly the repaid units leave msg.sender, the debtor's own wallet is never touched (so repaying on a borrower's behalf can never drain the borrower's standing token approval), and no third wallet's balance moves (checked with no repay callback) balance[loanToken][msg.sender] ' == balance[loanToken][msg.sender] - units AND onBehalf not in {msg.sender, Midnight} => balance[loanToken][onBehalf] ' == balance[loanToken][onBehalf] AND forall v not in {msg.sender, Midnight}. balance[loanToken][v] ' == balance[loanToken][v]	✓		
HL-MI-52	withdrawPaysReceiverExactly	a lender's withdrawal pays out to the designated receiver only — exactly the withdrawn units land in the receiver's wallet, and no other wallet's loan-token balance moves balance[loanToken][receiver] ' == balance[loanToken][receiver] + units AND forall v not in {receiver, Midnight}. balance[loanToken][v] ' == balance[loanToken][v]	✓		
HL-MI-53	supplyCollateralPullsSenderOnly	posting collateral pulls the tokens from the caller's wallet only — exactly the deposited assets leave msg.sender, the credited borrower's own wallet is never touched (no draining of the borrower's standing approval), and no third wallet's balance moves balance[collateralToken[i]][msg.sender] ' == balance[collateralToken[i]][msg.sender] - assets AND onBehalf not in {msg.sender, Midnight} => balance[collateralToken[i]][onBehalf] ' == balance[collateralToken[i]][onBehalf] AND forall v not in {msg.sender, Midnight}. balance[collateralToken[i]][v] ' == balance[collateralToken[i]][v]	✓		
HL-MI-54	withdrawCollateralPaysReceiverExactly	withdrawing collateral pays the designated receiver only — exactly the withdrawn assets land in the receiver's wallet, and no other wallet's collateral-token balance moves balance[collateralToken[i]][receiver] ' == balance[collateralToken[i]][receiver] + assets AND forall v not in {receiver, Midnight}. balance[collateralToken[i]][v] ' == balance[collateralToken[i]][v]	✓		
HL-MI-55	claimContinuousFeePaysReceiverExactly	when the fee claimer collects the protocol's continuous-fee credit (cfc — fee units accrued to the protocol), the payout reaches the designated receiver only — exactly the claimed amount lands there, and no other wallet's loan-token balance moves balance[loanToken][receiver] ' ==	✓		

		<pre>balance[loanToken][receiver] + amount AND forall v not in {receiver, Midnight}. balance[loanToken][v]' == balance[loanToken][v]</pre>			
HL-MI-56	<code>claimSettlementFeePaysReceiverExactly</code>	when the fee claimer collects accumulated settlement fees for a token, the payout reaches the designated receiver only — exactly the claimed amount lands there, and no other wallet's balance of that token moves <pre>balance[token][receiver]' == balance[token][receiver] + amount AND forall v not in {receiver, Midnight}. balance[token][v]' == balance[token][v]</pre>	✓		
HL-MI-22m	<code>gettersMatchStoragePerId</code>	public view functions keyed by a market id report exactly that market's stored accounting and never another market's — the market's total loan units (totalUnits), its withdrawable liquidity, and per-user lender credit and borrower debt all read back the storage of the queried market, checked across two distinct markets <pre>forall id in {idA, idB}. totalUnits(id) == totalUnits[id] AND withdrawable(id) == withdrawable[id] AND credit(idA, u) == credit[idA][u] AND debt(idB, u) == debt[idB][u]</pre>	✓		
HL-MI-35b	<code>rcfDustEscapeReachable</code>	the dust escape is a real code path — there exists a successful normal-mode liquidation whose repayment exceeds the recovery-close-factor cap <pre>satisfy: exists execution of liquidate. repaid > maxRepaid where maxRepaid = ceil((debt[borrower] - maxDebt) * WAD^2 / (WAD^2 - maxLif * lltv)); maxDebt = floor(floor(collateral[borrower][0] * price / ORACLE_PRICE_SCALE) * lltv / WAD) (pre-state)</pre>	✓		
HL-MI-35c	<code>rcfDustEscapeTwoCollateral</code>	the liquidation repayment cap stays sound when the borrower posts two collateral types — the cap counts the health value of BOTH collateral slots (so the second slot tightens, never widens, how much may be repaid), while the dust test that deactivates the RCF cap applies to the liquidated slot only; repaying beyond the cap is only possible when the liquidated slot's remaining value is dust (normal-mode repaid-input path, lltv < WAD) <pre>repaid > maxRepaid => max(0, dustValue_idx - maxRepaid) < rcfThreshold where maxDebt = Σ {i in {0,1}} floor(floor(collateral[borrower][i] * price_i / ORACLE_PRICE_SCALE) * lltv_i / WAD); dustValue_idx = floor(floor(collateral[borrower][idx] * price_idx / ORACLE_PRICE_SCALE) * WAD / maxLif_idx); maxRepaid = ceil((debt[borrower] - maxDebt) * WAD^2 / (WAD^2 - maxLif_idx * lltv_idx)) (pre- state)</pre>	⌚		timeout: liquidate
HL-MI-37b	<code>postSlashSolvencyPreservedExceptLiquidate</code>	outside liquidation the solvency bound is exact — every entry point other than liquidate keeps the realizable lender value (summed over the three modeled users) plus the continuous-fee credit within the market's total loan units with no rounding tolerance at all <pre>forall f != liquidate. realizableValue <= totalUnits => realizableValue' <= totalUnits' where realizableValue = Σ_u postSlashCredit(u) + continuousFeeCredit</pre>	⌚		timeout: take, withdraw
HL-MI-46b	<code>settlementFeeNeverExceedsProtocolMax</code>	the published settlement-fee schedule itself never exceeds the protocol's hard cap — for any time to maturity, the interpolated settlement-fee rate stays at or below the maximum rate (0.5% of the 1e18 fixed-point scale, WAD) <pre>forall ttm. settlementFee(id, ttm) <= MAX_SETTLEMENT_FEE_360_DAYS (== 0.005e18)</pre>	✓		

HL-MI-57	<code>liquidateCollateralTokenRoutingExact</code>	the collateral a liquidation seizes reaches the liquidator's designated receiver only — exactly the returned seized assets land in the receiver's wallet, the borrower's own wallet is never touched (the collateral leaves the protocol's custody, not the borrower's wallet), and no third wallet's collateral-token balance moves <pre>balance[collateralToken[i]][receiver]' == balance[collateralToken[i]][receiver] + seized AND borrower not in {receiver, Midnight} => balance[collateralToken[i]] [borrower]' == balance[collateralToken[i]] [borrower] AND forall v not in {receiver, Midnight}. balance[collateralToken[i]][v]' == balance[collateralToken[i]][v]</pre>	✓		
HL-MI-58	<code>liquidateLoanTokenRoutingExact</code>	a liquidation's repayment is pulled from the liquidator's side only — exactly the returned repaid units leave the resolved payer (the callback contract if one is given, otherwise the caller), the borrower's own wallet is never pulled (the debtor cannot be made to pay for their own liquidation through a standing approval), and no third wallet's loan-token balance moves <pre>balance[loanToken][payer]' == balance[loanToken][payer] - repaid AND borrower not in {payer, Midnight} => balance[loanToken][borrower]' == balance[loanToken][borrower] AND forall v not in {payer, Midnight}. balance[loanToken][v]' == balance[loanToken][v] where payer = (callback != 0 ? callback : msg.sender)</pre>	✓		
HL-MI-59	<code>badDebtFormulaExact</code>	the bad debt a liquidation socializes onto lenders is exactly the borrower's true shortfall — the drop in the market's total loan units equals the debt minus the worst-case recoverable value of ALL the borrower's collateral (each slot valued at its oracle price and discounted by its maxLif, rounded against the write-off), floored at zero; the formula involves no liquidator-chosen input, so the seized and repaid amounts cannot steer how much loss gets socialized (checked over the two modeled collateral slots) <pre>totalUnits - totalUnits' == max(0, debt[borrower] - Σ_{i in {0,1}} ceil(ceil(collateral[borrower][i] * price_i / ORACLE_PRICE_SCALE) * WAD / maxLif_i)) (RHS pre-state)</pre>	✓		
HL-MI-60	<code>seizureToRepaidConversionExact</code>	when the liquidator names the collateral to seize, the debt they must repay is computed exactly — the returned seized amount is the input verbatim, and the repaid units equal the seized collateral's oracle value divided by the time-ramped liquidation incentive factor (lif), with both division steps rounded against the liquidator so they can never under-pay per seized unit <pre>seized == seizedAssetsIn AND repaid == ceil(ceil(seizedAssetsIn * price / ORACLE_PRICE_SCALE) * WAD / lif) where lif = postMaturityMode ? min(maxLif, WAD + floor((maxLif - WAD) * max(0, now - maturity) / 3600)) : maxLif</pre>	✓		
HL-MI-61	<code>repaidToSeizedConversionExact</code>	when the liquidator names the debt to repay, the collateral they receive is computed exactly — the returned repaid amount is the input verbatim, and the seized collateral equals the repaid units scaled up by the time-ramped liquidation incentive factor (lif) and converted at the oracle price, with both steps rounded down so the borrower can never be over-seized per repaid unit <pre>repaid == repaidUnitsIn AND seized == floor(floor(repaidUnitsIn * lif / WAD) * ORACLE_PRICE_SCALE / price) where lif = postMaturityMode ? min(maxLif, WAD +</pre>	✓		

		<code>floor((maxLif - WAD) * max(0, now - maturity) / 3600)) : maxLif</code>			
HL-MI-62	<code>postMaturityLifIncentiveMonotoneInTime</code>	the post-maturity liquidation bonus only grows with time — for a fixed repayment, the collateral a liquidator seizes now is at least what the conversion formula would have granted at any earlier post-maturity instant, so liquidators are never pushed to wait out a decaying incentive while an underwater position sits unliquidated <pre>forall tEarlier in (maturity, block.timestamp]. seized >= floor(floor(repaidUnitsIn * lif(tEarlier) / WAD) * ORACLE_PRICE_SCALE / price) where lif(t) = min(maxLif, WAD + floor((maxLif - WAD) * (t - maturity) / 3600))</pre>			timeout: liquidate

Market Creation

The `touchMarket` creation branch is verified in its own conf ([market_creation.conf](#))

→ [midnight_market_creation.spec](#)), the only conf that

imports the base setup WITHOUT the touchMarket summary (the public-function internal summary intercepts the external entry, so creation is dead code everywhere else).

Property	Name	Description	Audit	Mitig	Notes
MC-MI-01	<code>touchMarketCreatesReachable</code>	market creation is reachable — calling <code>touchMarket</code> on a market that has never been created can bring it into existence, with its tick spacing (the granularity of price ticks at which offers can be placed and filled) set to the protocol default of 4 <code>satisfy: exists execution of touchMarket(market) with tickSpacing[market] == 0. tickSpacing[market]' == 4</code>	✓		
MC-MI-02	<code>creationValidatesMarketParams</code>	a market can only be created with sound parameters — if creation succeeds, every validation gate held: the maturity date is at most 100 years away, the collateral list is non-empty and within the protocol bound, collateral tokens are non-zero and strictly ascending, each collateral's loan-to-liquidation-value threshold (<code>lltv</code>) is a governance-enabled tier, each collateral's liquidation cursor (which fixes its <code>maxLif</code>) is a governance-enabled cursor, and the resulting <code>maxLif</code> stays within bounds (<code>maxLif <= 2WAD</code> , and <code>lltvmaxLif <= 0.999e18*WAD</code> unless <code>lltv == WAD</code>) (checked over the two modeled collateral slots) <code>tickSpacing[market] == 0 AND touchMarket(market) succeeds => maturity <= now + 100 years AND 1 <= len(collateralParams) <= MAX_COLLATERALS AND collateralParams[0].token != 0 AND isLltvEnabled(collateralParams[0].lltv) AND isLiquidationCursorEnabled(collateralParams[0].liquidationCursor) AND maxLif(lltv[0], cursor[0]) <= 2*WAD AND (lltv[0] == WAD OR lltv[0]*maxLif(lltv[0], cursor[0]) <= 0.999e18*WAD) AND (len(collateralParams) > 1 => the same for slot 1 with token[1] > token[0])</code>	✓		
MC-MI-03	<code>creationCopiesDefaultFees</code>	market creation copies the fee schedule configured for the market's loan token into the new market verbatim — all seven breakpoints of the settlement fee (the fee charged on trades and collected into a per-token claimable pot) and the continuous fee rate charged on outstanding debt — so the protocol cannot create a market with fees other than the defaults set by the fee setter <code>tickSpacing[market] == 0 AND touchMarket(market) succeeds => (forall k in 0..6. settlementFeeCbp[market][k]' == defaultSettlementFeeCbp[loanToken][k]) AND continuousFee[market]' == defaultContinuousFee[loanToken]</code>	✓		
MC-MI-04	<code>creationSetsTickSpacingDefault</code>	market creation initializes the tick spacing (the granularity of price ticks at which offers can be placed and filled) to exactly the protocol default of 4 — a wrong initial spacing would silently change which prices lenders and borrowers can trade at <code>tickSpacing[market] == 0 AND touchMarket(market) succeeds => tickSpacing[market]' == 4</code>	✓		
MC-MI-05	<code>touchMarketIdempotent</code>	market creation can only happen once — touching an already-existing market is a no-op, so a repeat call can never re-run creation and overwrite the market's tick spacing, fee schedule, or aggregate accounting (the market's total loan units, the cumulative bad-debt socialization factor, the loan tokens available for withdrawal, and the continuous-fee credit owed to the protocol) <code>after touchMarket(market), a second touchMarket(market) satisfies X' == X for X in {tickSpacing[market], settlementFeeCbp0[market], settlementFeeCbp6[market], continuousFee[market], totalUnits, lossFactor, withdrawable, continuousFeeCredit}</code>	✓		
MC-MI-06	<code>creationDoesNotTouchPositionsOrPots</code>	market creation writes only configuration and never moves money — no lender's credit units, no borrower's debt, no fee accrued on a position but not yet collected (<code>pendingFee</code>), none of the market aggregates (<code>totalUnits</code> , <code>lossFactor</code> , <code>withdrawable</code> , <code>continuousFeeCredit</code>), and no token's claimable settlement-fee pot changes when a market is created <code>tickSpacing[market] == 0 AND touchMarket(market) succeeds => forall u, token. credit[u]' == credit[u] AND debt[u]' == debt[u] AND pendingFee[u]' == pendingFee[u] AND totalUnits' == totalUnits AND lossFactor' == lossFactor AND withdrawable' == withdrawable AND continuousFeeCredit' == continuousFeeCredit AND claimableSettlementFee[token]' == claimableSettlementFee[token]</code>	✓		
MC-MI-07	<code>creationValidatesChainIdAndMidnight</code>	a market can only be created with a self-consistent identity — its embedded <code>chainId</code> must match the live chain and its embedded midnight address must be this contract, binding every created market's id to the deploying chain and instance (the replacement for the old <code>INITIAL_CHAIN_ID</code> immutable), so a market struct minted for another chain or another Midnight instance can never be brought into existence here <code>tickSpacing[market] == 0 AND touchMarket(market) succeeds => market.chainId == block.chainid AND market.midnight == address(this)</code>	?		new

State Transitions

Multi-variable co-transition properties: how two or more storage variables must change together (or one's change forces another's) across a single call. Proven in the one-market regime; the cross-market liquidate frame (ST-MI-17) is verified in the many-market regime.

Property	Name	Description	Audit	Mitig	Notes
ST-MI-01	<code>takePairsCreditAndDebtDirectionally</code>	a trade through the take() entry point (a buyer fills a maker's offer) moves each side's position in the right direction: the buyer — the party receiving credit — never picks up debt, while the seller — the party taking on the loan — never gains credit and never sheds debt; checked for two distinct counterparties <code>debt[buyer]' <= debt[buyer] AND credit[seller]' <= credit[seller] AND debt[seller]' >= debt[seller], where buyer = offer.buy ? offer.maker : taker and seller = the other counterparty</code>	✓		
ST-MI-02	<code>creditDecreaseDoesNotRaisePendingFee</code>	when a lender's credit shrinks, the fee accrued on the position but not yet collected (pendingFee) is burned proportionally with it, so a smaller position can never end up owing MORE fee; holds for every entry point except the take() trade path, where the fee pre-charged on newly bought credit can legitimately raise pendingFee even as the buyer's existing credit is reduced <code>forall f != take, u. credit[u]' < credit[u] => pendingFee[u]' <= pendingFee[u]</code>	✓		
ST-MI-03	<code>takeCannotIncreaseDebtPostMaturity</code>	once a market is past its maturity date, trading can no longer create new debt: a take() (a buyer fills a maker's offer) executed after maturity may not increase the debt of either counterparty — maker and taker debts can only stay flat or fall <code>block.timestamp > market.maturity => debt[offer.maker]' <= debt[offer.maker] AND debt[taker]' <= debt[taker]</code>	✓		
ST-MI-04	<code>liquidateRequiresBorrowerDebt</code>	a liquidator can never liquidate a debt-free position: every liquidate call that succeeds implies the targeted borrower held positive debt beforehand <code>liquidate(borrower) succeeds => debt[borrower] > 0</code>	✓		
ST-MI-05	<code>lossFactorIncreaseCoincidesWithTotalUnitsDecrease</code>	socializing bad debt always destroys loan units: whenever the cumulative bad-debt socialization factor (lossFactor) rises — a bad-debt slash against lenders — the market's total loan units (totalUnits) must strictly fall, reflecting the written-off debt <code>forall f. lossFactor' > lossFactor => totalUnits' < totalUnits</code>	✓		
ST-MI-06	<code>withdrawCollateralMatchesMidnightBalance</code>	withdrawing collateral is token-conservative: collateral removed from borrower positions leaves the protocol's own token holdings one-for-one — summed over the three modeled users, the recorded collateral at the given index changes by exactly the change in the protocol's balance of that collateral token (with the receiver outside the protocol itself) <code>Σ_u (collateral[u][idx]' - collateral[u][idx]) == balance[collateralToken[idx]][Midnight]' - balance[collateralToken[idx]][Midnight]</code>	✓		
ST-MI-07	<code>collateralOpsPreserveCreditDebtFeeSurface</code>	posting or withdrawing collateral is a pure collateral movement: it leaves every position's credit, debt, and uncollected fee (pendingFee) untouched, and leaves the market's total loan units (totalUnits), the loan tokens available for withdrawal (withdrawable), and the protocol's continuous-fee credit (cfc) unchanged <code>forall f, u. f in {supplyCollateral, withdrawCollateral} => credit[u]' == credit[u] AND debt[u]' == debt[u] AND pendingFee[u]' == pendingFee[u] AND totalUnits' == totalUnits</code>	✓		

		<pre>AND withdrawable' == withdrawable AND continuousFeeCredit' == continuousFeeCredit</pre>			
ST-MI-08	<code>creditSideChangeStampsAccrual</code>	no function can change a lender's money without bringing the position fully up to date in the same step: whenever any call changes a position's credit or its uncollected fee (<code>pendingFee</code>), it must also stamp the position's accrual time to the current block timestamp and re-sync its snapshot of the cumulative bad-debt socialization factor (<code>lossFactor</code>), so fee accrual and bad-debt slashing can never be skipped on a touched position <pre>forall f, u. (credit[u]' != credit[u] OR pendingFee[u]' != pendingFee[u]) => lastAccrual[u]' == block.timestamp AND lastLossFactor[u]' == lossFactor'</pre>	✓		
ST-MI-09	<code>claimableSettlementFeeDecreasesOnlyViaClaim</code>	the per-token pot of settlement fees collected from trades can only be drained by an explicit claim through <code>claimSettlementFee</code> — no trade, lending, collateral, or liquidation path can take money out of the fee claimer's pot <pre>forall f. claimableSettlementFee[token]' < claimableSettlementFee[token] => f == claimSettlementFee</pre>	✓		
ST-MI-10	<code>claimSettlementFeeMatchesBalance</code>	claiming settlement fees pays out exactly what it deducts: a claim of <code>amount</code> reduces both the per-token claimable settlement-fee pot and the protocol's own balance of that token by exactly that amount, so the fee claimer cannot extract more than the pot records (receiver outside the protocol itself) <pre>claimableSettlementFee[token]' == claimableSettlementFee[token] - amount AND balance[token][Midnight]' == balance[token] [Midnight] - amount</pre>	✓		
ST-MI-11	<code>takeLeavesSellerLockedOrHealthy</code>	a trade can never leave its seller exposed to liquidation: every successful <code>take()</code> (a buyer fills a maker's offer) exits with the seller either healthy (collateral still covers the debt under the loan-to-liquidation-value threshold) or protected by the in-transaction liquidation lock <pre>liquidationLocked[id][seller]' OR isHealthy(market, seller)'</pre>	✓		
ST-MI-12	<code>tickSpacingRefinesToDivisor</code>	a market's tick spacing — the granularity at which offer prices may be quoted — can only be refined, never coarsened: any change must install a positive value that exactly divides the old one, so every previously valid price tick remains valid <pre>forall f. tickSpacing[id]' != tickSpacing[id] => tickSpacing[id]' > 0 AND tickSpacing[id] % tickSpacing[id]' == 0</pre>	✓		
ST-MI-13	<code>liquidateRequiresUnlockedBorrower</code>	liquidation respects the in-transaction liquidation lock: a liquidator can never liquidate a borrower whose position is currently liquidation-locked, even when liquidate is entered mid-transaction (e.g. from a trade callback) while the lock is still set <pre>liquidate(borrower) succeeds => NOT liquidationLocked[id][borrower]</pre>	✓		
ST-MI-14	<code>lossFactorRaisedOnlyByLiquidate</code>	only liquidation can socialize losses onto lenders: the cumulative bad-debt socialization factor (<code>lossFactor</code>), which lazily slashes every lender position on its next touch, can be raised by <code>liquidate</code> and by no other function — no fee, trade, or admin path can dilute lenders' credit <pre>forall f. lossFactor' > lossFactor => f == liquidate</pre>	✓		
		liquidation settles purely on the debt-and-collateral side: it never writes any position's credit-side accounting — credit, uncollected fee (<code>pendingFee</code>), accrual timestamp, or loss-factor snapshot — not even the liquidated			

ST-MI-15	<code>liquidatePreservesCreditSideSurface</code>	borrower's, because the bad-debt slash is applied lazily on each position's next touch; nor does it touch the market's fee configuration (the settlement-fee schedule, the continuous fee rate, or the tick spacing) <pre>forall u. credit[u]' == credit[u] AND pendingFee[u]' == pendingFee[u] AND lastAccrual[u]' == lastAccrual[u] AND lastLossFactor[u]' == lastLossFactor[u] AND tickSpacing' == tickSpacing AND continuousFee' == continuousFee AND forall bucket in 0..6. settlementFee[bucket]' == settlementFee[bucket]</pre>	✓		
ST-MI-16	<code>debtDecreaseOnlyViaTakeRepayOrLiquidate</code>	debt can only shrink through a legitimate repayment channel: a position's debt decreases only via the take() trade entry point (a buyer's purchase is netted against the buyer's existing debt), repay, or liquidate (the liquidator repays the debt and any shortfall is written off as bad debt) — withdrawals, fee claims, collateral operations, flash loans, and admin setters can never lower anyone's debt <pre>forall f, u. debt[u]' < debt[u] => f in {take, repay, liquidate}</pre>	✓		
ST-MI-17	<code>liquidateMarketIsolationMany</code>	liquidation is strictly market-local: a liquidator repaying debt and seizing collateral on one market leaves every other market completely untouched — both the other market's state (total loan units, withdrawable liquidity, bad-debt socialization factor, fee credit, and all fee parameters) and every user position on it; losses and funds can never leak across markets through a liquidation <pre>forall other != toId(market), u. every MarketState[other] field' == MarketState[other] field AND every position[other][u] field' == position[other] [u] field</pre>	✓		
ST-MI-18	<code>lltvEnabledIsMonotone</code>	governance can only widen the set of enabled LLTV tiers, never shrink it: across every entry point, an LLTV tier that is enabled before a call stays enabled after it (enableLltv only flips false → true; there is no disable path), so markets created against a tier can never be invalidated by a later governance action <pre>forall f, lltv. isLltvEnabled[lltv] => isLltvEnabled[lltv]'</pre>	?		new
ST-MI-19	<code>liquidationCursorEnabledIsMonotone</code>	governance can only widen the set of enabled liquidation cursors, never shrink it: across every entry point, a cursor that is enabled before a call stays enabled after it (enableLiquidationCursor only flips false → true) <pre>forall f, cursor. isLiquidationCursorEnabled[cursor] => isLiquidationCursorEnabled[cursor]'</pre>	?		new

Reachability

`satisfy()` rules that prove a meaningful non-reverting execution path EXISTS from a valid state. Their value is **anti-vacuity**: the verification model is heavily narrowed (one market, 3 users, 2 collateral slots, 5-tick price, oracle ≥ 1 , empty callbacks, all VS-MI loaded), and if that narrowing made a critical state unreachable, every ST/HL/bug-hunt rule quantifying over it would pass **vacuously**. An `UNSAT` here is a finding (the dependent rules are vacuous).

ID	Name	Proves reachable	Audit	Mitig
RC-		a lender can actually acquire credit through trading: there is a real execution of the take() trade entry point (a buyer fills a maker's offer) in which the buyer's interest-bearing credit balance strictly increases,		

MI-01	takeMintsCreditReachable	proving the credit-minting side of a trade is live <code>satisfy: exists execution of take.</code> <code>credit[buyer]' > credit[buyer]</code> , where <code>buyer = offer.maker</code> if the offer is a buy offer, else the taker	✓	
RC-MI-02	takeMintsDebtReachable	a borrower can actually take on debt through trading: there is a real execution of the take() trade entry point (a buyer fills a maker's offer) in which the seller's debt strictly increases, proving the borrow side of a trade is live <code>satisfy: exists execution of take.</code> <code>debt[seller]' > debt[seller]</code> , where <code>seller = the taker</code> if the offer is a buy offer, else <code>offer.maker</code>	✓	
RC-MI-03	takeCapturesSettlementFeeReachable	the protocol can actually earn trading fees: there is a real execution of the take() trade entry point (a buyer fills a maker's offer) in which the per-token pot of settlement fees claimable by the fee claimer strictly grows <code>satisfy: exists execution of take.</code> <code>claimableSettlementFee[loanToken]' > claimableSettlementFee[loanToken]</code>	✓	
RC-MI-04	withdrawReachable	a lender can actually withdraw: there is a real execution of withdraw that strictly reduces the lender's interest-bearing credit balance, proving the basic exit path for deposited funds is live <code>satisfy: exists execution of withdraw.</code> <code>credit[onBehalf]' < credit[onBehalf]</code>	✓	
RC-MI-05	withdrawFullCreditExitReachable	full-exit liveness for lenders: a lender holding a positive credit position can withdraw it down to exactly zero in a single call, so deposited funds are never structurally trapped in the market <code>satisfy: exists execution of withdraw.</code> <code>credit[onBehalf] > 0 AND credit[onBehalf]' == 0</code>	✓	
RC-MI-06	repayReachable	a borrower can actually repay: there is a real execution of repay that strictly reduces the borrower's debt, proving the basic debt-reduction path is live <code>satisfy: exists execution of repay.</code> <code>debt[onBehalf]' < debt[onBehalf]</code>	✓	
RC-MI-07	repayFullDebtReachable	full-exit liveness for borrowers: a borrower with positive debt can repay it down to exactly zero in a single call, so a debt position can always be fully closed <code>satisfy: exists execution of repay.</code> <code>debt[onBehalf] > 0 AND debt[onBehalf]' == 0</code>	✓	
RC-MI-08	supplyCollateralActivatesSlotReachable	collateral posting is live: a borrower can deposit collateral into a slot that currently holds nothing, turning it into an active (non-zero) collateral balance <code>satisfy: exists execution of supplyCollateral.</code> <code>collateral[onBehalf][i] == 0 AND collateral[onBehalf][i]' > 0</code>	✓	
		collateral recovery is live: a borrower can execute		

RC-MI-09	<code>withdrawCollateralReachable</code>	withdrawCollateral and strictly reduce one of their collateral balances (the call only succeeds while the borrower remains healthy, so this also shows the health check is not blanket-blocking withdrawals) <code>satisfy: exists execution of withdrawCollateral. collateral[onBehalf][i]' < collateral[onBehalf][i]</code>	✓	
RC-MI-10	<code>liquidateNormalModeReachable</code>	pre-maturity liquidation is live: a liquidator can execute a normal-mode liquidation (<code>postMaturityMode = false</code>) that strictly reduces an unhealthy borrower's debt, repaying it in exchange for seized collateral <code>satisfy: exists execution of liquidate(postMaturityMode = false). debt[borrower]' < debt[borrower]</code>	✓	
RC-MI-11	<code>liquidatePostMaturityReachable</code>	post-maturity liquidation is live: once the market's maturity has passed, a liquidator can execute a post-maturity-mode liquidation that strictly reduces the borrower's debt <code>satisfy: exists execution of liquidate(postMaturityMode = true) with block.timestamp > maturity. debt[borrower]' < debt[borrower]</code>	✓	
RC-MI-12	<code>liquidateRealizesBadDebtReachable</code>	bad-debt realization is live: a liquidation can exhaust the borrower's collateral and leave a shortfall, strictly increasing the cumulative bad-debt socialization factor (<code>lossFactor</code>) that spreads the loss across lenders <code>satisfy: exists execution of liquidate. lossFactor' > lossFactor</code>	✓	
RC-MI-13	<code>positionCanBeUnhealthy</code>	insolvency risk is representable: from a valid market state there exists a borrower whose collateral value no longer covers their debt, i.e. the protocol's health check can actually fail — without this, no borrower could ever qualify for liquidation and every liquidation property would hold trivially <code>satisfy: exists valid state. isHealthy(market, borrower) == false</code>	✓	
RC-MI-14	<code>claimSettlementFeeReachable</code>	the settlement-fee pot can actually be paid out: the fee claimer can execute <code>claimSettlementFee</code> and strictly reduce the per-token pot of trading fees accumulated by the protocol <code>satisfy: exists execution of claimSettlementFee. claimableSettlementFee[token]' < claimableSettlementFee[token]</code>	✓	
RC-MI-15	<code>claimContinuousFeeReachable</code>	the continuous-fee pot can actually be paid out: the fee claimer can execute <code>claimContinuousFee</code> and strictly reduce the continuous-fee credit (<code>cfc</code>), the fee units accrued to the protocol from outstanding debt <code>satisfy: exists execution of claimContinuousFee. continuousFeeCredit' < continuousFeeCredit</code>	✓	
		flash loans are live: a <code>flashLoan</code> call can complete with		

RC-MI-16	<code>flashLoanReachable</code>	the protocol's token balance exactly restored, confirming the zero-fee borrow-and-return path works end to end <code>satisfy: exists execution of flashLoan.</code> <code>balance[token][Midnight]' == balance[token][Midnight]</code>	✓	
RC-MI-17	<code>updatePositionSlashesCreditReachable</code>	lazy loss socialization is live: touching a position via <code>updatePosition</code> can strictly reduce the holder's credit, i.e. bad debt that was previously socialized through the cumulative bad-debt socialization factor (<code>lossFactor</code>) can actually be charged to a lender's position when it is next touched <code>satisfy: exists execution of updatePosition.</code> <code>credit[user]' < credit[user]</code>	✓	
RC-MI-18	<code>borrowThenLiquidateReachable</code>	the full borrow-to-liquidation lifecycle is live end to end: a single scenario exists in which a seller takes on positive debt through the <code>take()</code> trade entry point (a buyer fills a maker's offer) and that same debt is then strictly reduced by a subsequent liquidation of the seller <code>satisfy: exists execution of take;</code> <code>liquidate(borrower = seller). debt[seller]_mid > 0 AND debt[seller]' < debt[seller]_mid</code>, where <code>debt[seller]_mid</code> is the seller's debt after <code>take</code> and <code>'</code> is the state after <code>liquidate</code>	✓	

Reverts

`@withrevert` + `assert(condition => lastReverted)` rules — the dual of reachability: they prove a function MUST revert under a disallowed condition (access control = anti-theft, input validation, state preconditions). Each rule is single-call with a condition over hooked ghosts, so the prover cannot havoc an untracked slot into a spurious revert; Midnight has no pause.

ID	Name	Reverts when	Audit	Mitig
RV-MI-01	<code>setConfiguratorRevertsWhenNotConfigurator</code>	only the protocol's role administrator (the configurator) can hand the role-administration power to a new account: a call to <code>setConfigurator</code> from any other address is rejected unconditionally, so control over all protocol roles cannot be hijacked <code>e.msg.sender != configurator =></code> <code>reverts(setConfigurator(newConfigurator))</code>	✓	
RV-MI-02	<code>setFeeSetterRevertsWhenNotConfigurator</code>	only the protocol's role administrator (the configurator) can appoint the account that controls fee parameters (the feeSetter): a call to <code>setFeeSetter</code> from any other address is rejected unconditionally <code>e.msg.sender != configurator =></code> <code>reverts(setFeeSetter(newFeeSetter))</code>	✓	
RV-MI-03	<code>setFeeClaimerRevertsWhenNotConfigurator</code>	only the protocol's role administrator (the configurator) can appoint the account entitled to collect accrued protocol fees (the feeClaimer): a call to <code>setFeeClaimer</code> from any other address is rejected unconditionally, so the right to protocol fee revenue cannot be redirected <code>e.msg.sender != configurator =></code> <code>reverts(setFeeClaimer(newFeeClaimer))</code>	✓	
RV-MI-04	<code>setTickSpacingSetterRevertsWhenNotConfigurator</code>	only the protocol's role administrator (the configurator) can appoint the account that controls the price granularity of market offers (the tickSpacingSetter): a call to <code>setTickSpacingSetter</code> from any other address is rejected unconditionally <code>e.msg.sender != configurator =></code> <code>reverts(setTickSpacingSetter(newTickSpacingSetter))</code>	✓	
RV-MI-05	<code>setMarketSettlementFeeRevertsWhenNotFeeSetter</code>	only the fee administrator (the feeSetter) can change a market's settlement fee — the fee charged on trades and paid into the protocol's per-token claimable pot: a call to <code>setMarketSettlementFee</code> from any other address is rejected unconditionally	✓	

		<pre>e.msg.sender != feeSetter => reverts(setMarketSettlementFee(id, index, newFee))</pre>		
RV-MI-06	<code>setMarketContinuousFeeRevertsWhenNotFeeSetter</code>	only the fee administrator (the feeSetter) can change a market's continuous fee — the rate at which fee units accrue to the protocol on outstanding borrower debt: a call to setMarketContinuousFee from any other address is rejected unconditionally <pre>e.msg.sender != feeSetter => reverts(setMarketContinuousFee(id, newFee))</pre>	✓	
RV-MI-07	<code>setMarketTickSpacingRevertsWhenNotTickSpacingSetter</code>	only the designated tickSpacingSetter can change a market's tick spacing — the price granularity at which offers may be quoted: a call to setMarketTickSpacing from any other address is rejected unconditionally <pre>e.msg.sender != tickSpacingSetter => reverts(setMarketTickSpacing(id, newTickSpacing))</pre>	✓	
RV-MI-08	<code>claimSettlementFeeRevertsWhenNotFeeClaimer</code>	only the designated fee collector (the feeClaimer) can withdraw accumulated settlement fees from the protocol's per-token pot: a call to claimSettlementFee from any other address is rejected unconditionally, so protocol fee revenue cannot be stolen <pre>e.msg.sender != feeClaimer => reverts(claimSettlementFee(token, amount, receiver))</pre>	✓	
RV-MI-09	<code>claimContinuousFeeRevertsWhenNotFeeClaimer</code>	only the designated fee collector (the feeClaimer) can withdraw the continuous-fee credit (cfc) — fee units accrued to the protocol from interest on borrower debt: a call to claimContinuousFee from any other address is rejected unconditionally <pre>e.msg.sender != feeClaimer => reverts(claimContinuousFee(market, amount, receiver))</pre>	✓	
RV-MI-10	<code>withdrawRevertsWhenUnauthorized</code>	nobody can pull loan tokens out of another lender's position: withdrawing on behalf of an account reverts unless the caller is that account or a delegate it has approved via isAuthorized <pre>e.msg.sender != onBehalf AND NOT isAuthorized[onBehalf] [e.msg.sender] => reverts(withdraw(market, units, onBehalf, receiver))</pre>	✓	
RV-MI-11	<code>withdrawCollateralRevertsWhenUnauthorized</code>	nobody can take collateral a borrower has posted: withdrawing collateral on behalf of an account reverts unless the caller is that account or a delegate it has approved via isAuthorized <pre>e.msg.sender != onBehalf AND NOT isAuthorized[onBehalf] [e.msg.sender] => reverts(withdrawCollateral(market, collateralIndex, assets, onBehalf, receiver))</pre>	✓	
RV-MI-12	<code>repayRevertsWhenUnauthorized</code>	repaying a borrower's debt is restricted to the borrower themselves or a delegate they have approved via isAuthorized: a repay attempt on behalf of another account by anyone else reverts, so third parties cannot manipulate someone else's debt position <pre>e.msg.sender != onBehalf AND NOT isAuthorized[onBehalf] [e.msg.sender] => reverts(repay(market, units, onBehalf, callback, data))</pre>	✓	
RV-MI-13	<code>supplyCollateralRevertsWhenUnauthorized</code>	adding collateral to another user's position requires that user's consent: supplyCollateral on behalf of an account reverts unless the caller is that account or a delegate it has approved via isAuthorized, so nobody can alter someone else's collateral profile uninvited <pre>e.msg.sender != onBehalf AND NOT isAuthorized[onBehalf] [e.msg.sender] => reverts(supplyCollateral(market, collateralIndex, assets, onBehalf))</pre>	✓	
RV-MI-14	<code>setIsAuthorizedRevertsWhenUnauthorized</code>	delegation rights cannot be self-granted: only the account itself, or a delegate it has already approved via isAuthorized, can change who is authorized to act on its positions — any other caller's attempt to rewrite an account's authorizations reverts, ruling out privilege escalation <pre>e.msg.sender != onBehalf AND NOT isAuthorized[onBehalf] [e.msg.sender] => reverts(setIsAuthorized(authorized, newIsAuthorized, onBehalf))</pre>	✓	
RV-MI-15	<code>takeRevertsWhenTakerUnauthorized</code>	nobody can execute a trade in someone else's name through the take() trade entry point (a buyer fills a maker's offer): a call naming an account as the taker reverts unless the caller is that account or a delegate it has approved via isAuthorized <pre>e.msg.sender != taker AND NOT isAuthorized[taker] [e.msg.sender] => reverts(take(offer, ..., taker, ...))</pre>	✓	
RV-MI-16	<code>liquidateRevertsWhenNotBorrower</code>	a user who owes nothing cannot be liquidated: any liquidate call targeting a borrower with zero outstanding debt reverts, so a liquidator can never seize collateral from a debt-free account <pre>debt[borrower] == 0 => reverts(liquidate(market, ..., borrower, ...))</pre>	✓	

RV-MI-17	<code>liquidateRevertsOnInconsistentInput</code>	a liquidator must name the trade by exactly one side: either the amount of collateral to seize (<code>seizedAssets</code>) or the amount of debt to repay (<code>repaidUnits</code>) — the other is derived by the protocol; specifying both at once is ambiguous and the call reverts <pre>seizedAssets != 0 AND repaidUnits != 0 => reverts(liquidate(market, collateralIndex, seizedAssets, repaidUnits, ...))</pre>	✓	
RV-MI-18	<code>takeRevertsOnSelfTake</code>	a maker cannot trade with themselves: the <code>take()</code> trade entry point (a buyer fills a maker's offer) rejects any fill in which the taker is the same account as the offer's maker, preventing self-dealing wash trades <pre>offer.maker == taker => reverts(take(offer, ..., taker, ...))</pre>	✓	
RV-MI-19	<code>takeRevertsOnBothCapsNonZero</code>	an offer must bound its size in exactly one denomination — either a cap in loan-token assets (<code>maxAssets</code>) or a cap in loan units (<code>maxUnits</code>): the <code>take()</code> trade entry point (a buyer fills a maker's offer) reverts on any offer that sets both caps at once (<code>InvalidOfferCaps</code>) <pre>offer.maxAssets != 0 AND offer.maxUnits != 0 => reverts(take(offer, ...))</pre>	✓	
RV-MI-20	<code>setConsumedRevertsOnNonMonotone</code>	the per-group consumed counter — which tracks how much of a maker's signed offer quota has already been filled — can only move forward: even an account's owner or approved delegate cannot rewind it below its current value, so spent offer capacity can never be restored <pre>amount < consumed[onBehalf][group] => reverts(setConsumed(group, amount, onBehalf))</pre>	✓	
RV-MI-21	<code>coreViewsNeverRevert</code>	the core read-only getters always answer: in any valid protocol state, querying a lender's credit units, a borrower's debt, the market's total loan units (<code>totalUnits</code>), or the loan tokens currently available for withdrawal from the market (<code>withdrawable</code>) can never revert, so off-chain integrators and on-chain callers can always read positions and market totals <pre>NOT reverts(credit(id, user)) AND NOT reverts(debt(id, user)) AND NOT reverts(totalUnits(id)) AND NOT reverts(withdrawable(id))</pre>	✓	
RV-MI-22	<code>enableLltvRevertsWhenNotConfigurator</code>	only the configurator can enable a new LLTV tier: <code>enableLltv</code> from any other address is rejected unconditionally, so the set of borrowable loan-to-liquidation thresholds cannot be widened by an unauthorized caller <pre>e.msg.sender != configurator => reverts(enableLltv(lltv))</pre>	?	
RV-MI-23	<code>enableLiquidationCursorRevertsWhenNotConfigurator</code>	only the configurator can enable a new liquidation cursor: <code>enableLiquidationCursor</code> from any other address is rejected unconditionally <pre>e.msg.sender != configurator => reverts(enableLiquidationCursor(liquidationCursor))</pre>	?	
RV-MI-24	<code>enableLltvRevertsOnLltvAboveWad</code>	an LLTV tier above 100% (WAD) is nonsensical and cannot be enabled: even the configurator's <code>enableLltv</code> reverts when <code>lltv > WAD</code>, so <code>maxLif</code>'s denominator stays well-defined for every tier <pre>lltv > WAD => reverts(enableLltv(lltv))</pre>	?	
RV-MI-25	<code>enableLiquidationCursorRevertsOnCursorAtOrAboveWad</code>	a liquidation cursor must be strictly below 100% (WAD) so that <code>maxLif</code>'s denominator stays positive for every enabled <code>lltv</code>: even the configurator's <code>enableLiquidationCursor</code> reverts when <code>liquidationCursor >= WAD</code> <pre>liquidationCursor >= WAD => reverts(enableLiquidationCursor(liquidationCursor))</pre>	?	
RV-MI-26	<code>takeRevertsOnContinuousFeeAboveOfferCap</code>	a maker buyer is protected against future continuous-fee increases: <code>take()</code> reverts when the market's current continuous fee exceeds the offer's <code>continuousFeeCap</code>, so an offer can never be filled at a continuous fee higher than the maker agreed to <pre>continuousFee(toId(offer.market)) > offer.continuousFeeCap => reverts(take(offer, ...))</pre>	?	
RV-MI-27	<code>takeRevertsOnUnusedReceiverNonZero</code>	the unused settlement receiver must be left zero: <code>take()</code> reverts if the offer is a buy and <code>offer.receiverIfMakerIsSeller</code> is non-zero, or if the offer is a sell and <code>receiverIfTakerIsSeller</code> is non-zero, guarding against silently mis-routed seller proceeds <pre>(offer.buy ? offer.receiverIfMakerIsSeller != 0 : receiverIfTakerIsSeller != 0) => reverts(take(offer, ...))</pre>	?	
RV-MI-28	<code>takeRevertsOnBothCapsZero</code>	an offer must bound its size in exactly one denomination — leaving BOTH <code>maxAssets</code> and <code>maxUnits</code> at zero is rejected by <code>take()</code> (the complement of RV-MI-19's both-non-zero case) <pre>offer.maxAssets == 0 AND offer.maxUnits == 0 => reverts(take(offer, ...))</pre>	?	

Access Control

Parametric rules asserting that a role-gated or authorization-gated variable changes only when the caller holds the role / is authorized (state-change $\Rightarrow$ role). The robust dual of the revert rules: a selector-anchored revert rule misses a renamed or second setter, whereas this ghost-state-change form catches any function that writes the gated variable. Midnight uses address roles, so the check is `msg.sender == <role>`.

ID	Name	Gated variable $\Rightarrow$ required caller	Audit	Mitig
AC-MI-01	<code>onlyConfiguratorChangesConfigurator</code>	the configurator is the protocol's governance admin, the address that appoints every other role: no matter which function is called, the configurator address can only be rotated by the current configurator itself, so protocol governance cannot be hijacked by any other caller <code>forall f. configurator' != configurator => e.msg.sender == configurator</code>	✓	
AC-MI-02	<code>onlyConfiguratorChangesFeeSetter</code>	the feeSetter is the role that configures the protocol's fee rates: across every entry point, the feeSetter address can only be replaced when the caller is the current configurator (governance), so no other party can install a fee-setting authority <code>forall f. feeSetter' != feeSetter => e.msg.sender == configurator</code>	✓	
AC-MI-03	<code>onlyConfiguratorChangesFeeClaimer</code>	the feeClaimer is the only role allowed to withdraw the protocol's accrued fee revenue: across every entry point, the feeClaimer address can only be replaced when the caller is the current configurator (governance), so no other party can redirect fee revenue to itself <code>forall f. feeClaimer' != feeClaimer => e.msg.sender == configurator</code>	✓	
AC-MI-04	<code>onlyConfiguratorChangesTickSpacingSetter</code>	the tickSpacingSetter is the role that controls the granularity of the price grid on which borrower offers are placed: across every entry point, the tickSpacingSetter address can only be replaced when the caller is the current configurator (governance) <code>forall f. tickSpacingSetter' != tickSpacingSetter => e.msg.sender == configurator</code>	✓	
AC-MI-05	<code>onlyFeeSetterChangesDefaultSettlementFee</code>	the default settlement-fee schedule is the per-loan-token table of trade fees (one bucket per time-to-maturity band) that newly created markets inherit; across every entry point, a default settlement-fee bucket can only change when the caller is the feeSetter, so nobody else can raise fees on traders or zero out protocol revenue <code>forall f. defaultSettlementFeeCbp[token][index]' != defaultSettlementFeeCbp[token][index] => e.msg.sender == feeSetter'</code>	✓	
AC-MI-06	<code>onlyFeeSetterChangesDefaultContinuousFee</code>	the default continuous fee is the per-loan-token rate at which borrower debt accrues fee units to the protocol over time, inherited by newly created markets; across every entry point, it can only change when the caller is the feeSetter, so nobody else can reprice the cost of borrowing <code>forall f. defaultContinuousFee[token]' != defaultContinuousFee[token] => e.msg.sender == feeSetter'</code>	✓	
		a live market carries seven settlement-fee buckets that determine the trade fee charged on take() fills (the take())		

AC-MI-07	<code>onlyFeeSetterChangesMarketSettlementFee</code>	trade entry point, where a buyer fills a maker's offer) by time to maturity: across every entry point, if any of the seven buckets changes, the caller must be the feeSetter — nobody else can reprice trading fees on an existing market <pre>forall f. (exists i in 0..6. settlementFeeCbp_i' != settlementFeeCbp_i) => e.msg.sender == feeSetter'</pre>		
AC-MI-08	<code>onlyFeeSetterChangesMarketContinuousFee</code>	a live market's continuous fee is the rate at which outstanding borrower debt accrues fee units to the protocol over time: across every entry point, it can only change when the caller is the feeSetter, so nobody else can change what borrowers pay on an existing market <pre>forall f. continuousFee' != continuousFee => e.msg.sender == feeSetter'</pre>		
AC-MI-09	<code>onlyTickSpacingSetterChangesTickSpacing</code>	tick spacing is the granularity of the price grid on which borrower offers may sit in a market: across every entry point, the market's tick spacing can only change when the caller is the tickSpacingSetter, so nobody else can alter where offers may be priced <pre>forall f. tickSpacing' != tickSpacing => e.msg.sender == tickSpacingSetter'</pre>		
AC-MI-10	<code>onlyFeeClaimerDrainsClaimableSettlementFee</code>	claimableSettlementFee[token] is the per-token pot of trade fees owed to the protocol, and trading only adds to it: across every entry point, the pot can only decrease — i.e. fee revenue can only be paid out — when the caller is the feeClaimer, so nobody else can drain protocol fees <pre>forall f. claimableSettlementFee[token]' < claimableSettlementFee[token] => e.msg.sender == feeClaimer'</pre>		
AC-MI-11	<code>onlyAuthorizerChangesAuthorization</code>	a user <code>a</code> may delegate management of their positions to another address through the authorization graph (isAuthorized): the delegation flag isAuthorized[a][b] can only be flipped by <code>a</code> itself or by an address that <code>a</code> had already authorized before the call — a third party can never grant itself (or anyone else) control over a's funds <pre>forall f. isAuthorized[a][b]' != isAuthorized[a][b] => (e.msg.sender == a OR isAuthorized[a][e.msg.sender])</pre>		
AC-MI-12	<code>onlyConfiguratorChangesLltvEnabled</code>	the set of enabled LLTV tiers (the loan-to-liquidation thresholds at which markets may be created) is governance-controlled: across every entry point, an LLTV tier's enabled flag can only change when the caller is the configurator, so nobody else can widen the borrowable-risk surface <pre>forall f, lltv. isLltvEnabled[lltv]' != isLltvEnabled[lltv] => e.msg.sender == configurator</pre>		
AC-MI-13	<code>onlyConfiguratorChangesLiquidationCursorEnabled</code>	the set of enabled liquidation cursors (which fix each collateral's maxLif at market creation) is governance-controlled: across every entry point, a cursor's enabled flag can only change when the caller is the configurator <pre>forall f, cursor. isLiquidationCursorEnabled[cursor]' != isLiquidationCursorEnabled[cursor] => e.msg.sender == configurator</pre>		

Gates

Enter-gate / liquidator-gate / ratifier enforcement on the `take` and `liquidate` paths, enabled by recording gate summaries ([setup/gates.spec](#)) that pin each gate to at most one consultation per entry. Conf: [gates.conf](#).

Property	Name	Description	Audit	Mitig	Notes
GT-MI-01	<code>takeBuyerCreditIncreaseRequiresGateApproval</code>	on a market protected by an enter gate, the <code>take()</code> trade entry point (a buyer fills a maker's offer) can increase the buyer's lender credit only if the gate contract was consulted for that buyer and approved — no one can enter a gated market on the lending side without the gate's consent; the credit increase is measured net of the lazy fee accrual and bad-debt slashing that <code>take()</code> first realizes into the buyer's position <code>credit[buyer]' > creditAfterAccrualAndSlash[buyer]</code> <code>AND enterGate != 0 =></code> <code>enterGate.canIncreaseCredit(buyer)</code> was called AND returned <code>true</code>	✓		
GT-MI-02	<code>takeSellerDebtIncreaseRequiresGateApproval</code>	on a market protected by an enter gate, <code>take()</code> can increase the seller's debt (open or grow a borrow position) only if the gate contract was consulted for that seller and approved — no one can take on new debt in a gated market without the gate's consent <code>debt[seller]' > debt[seller] AND enterGate != 0 =></code> <code>enterGate.canIncreaseDebt(seller)</code> was called AND returned <code>true</code>	✓		
GT-MI-03	<code>liquidateRequiresLiquidatorGateApproval</code>	on a market protected by a liquidator gate, a liquidation (the liquidator repays a borrower's debt and seizes collateral) can succeed only if the gate contract was consulted for the caller and approved — unapproved liquidators cannot seize collateral on gated markets <code>liquidate succeeds AND liquidatorGate != 0 =></code> <code>liquidatorGate.canLiquidate(msg.sender)</code> was called AND returned <code>true</code>	✓		
GT-MI-04	<code>takeRequiresRatifierSuccess</code>	every trade settled through <code>take()</code> must first be ratified: the offer's designated ratifier contract is consulted, and the trade succeeds only when that contract returns the protocol's <code>CALLBACK_SUCCESS</code> magic value — no <code>take()</code> path can fill an offer while skipping the ratifier check <code>take succeeds => offer.ratifier.isRatified(offer,</code> <code>ratifierData) ==</code> <code>keccak256("morpho.midnight.callbackSuccess")</code>	✓		

Real Issues Found

This section documents the findings surfaced by the formal properties. Each issue references the rule that detected it; the property should pass after the fix (Mitig column).

[INFO] The continuous-fee pot can transiently exceed the withdrawable pool

The conjecture `cfc +  $\sum$  debt <= totalUnits` (equivalent to `cfc <= withdrawable` under VS-MI-17) is refuted on `updatePosition` / `withdraw`: states are reachable where the accrued continuous-fee credit exceeds the withdrawable pool, so `claimContinuousFee` temporarily reverts on its guarded underflow (src L318-320) until debt is repaid. The state is ordinary market life, not an attack: `take` lends peer-to-peer without touching `withdrawable` (only `repay` and `liquidate` replenish it) while the fee accrues continuously, so mid-term the fee pot can exceed the repaid pool. No funds at risk: the claim takes an explicit amount (`min(cfc, withdrawable)` always succeeds), the entitlement is conserved exactly (HL-MI-23), bad debt slashes the pot proportionally with lenders (HL-MI-31), and once all debt is repaid `withdrawable == totalUnits`

`= cfc`` (VS-MI-17/18) makes the full claim serviceable — a liveness consideration for fee-claimer integrations only.

Detected by `continuousFeeCreditWithinTotalUnitsMinusDebt` (VS-MI-22).

Recommendation: none required on-chain — the shared-pool race is already documented in the protocol header (src L27-29: lenders "and the fee claimer" may race for assets that become withdrawable before maturity); integrators should claim `min(cfc, withdrawable)` and retry as repayments arrive.

Mitigation: —

Verification Results

A total of **167 properties** across eight categories were executed per-rule against the audit commit (`morpho-org/midnight@7538c43`) on the local Certora Prover (certora-cli 8.13.0, solc 0.8.34), covering all 11 property confs — including the one-market, take-only, and many-market valid-state regimes. Per-rule verdicts with result links live in [fv_docs/run_report.md](#) (the single source of truth).

Category	Result
Valid State	17 · 4 · 1 (22)
High-Level	55 · 12 (67)
State Transitions	17
Access Control	12
Gates	4
Market Creation	6
Reachability	18 — every <code>satisfy</code> witnessed, no vacuous rule
Reverts	21
Total	150 · 16 · 1 (167)

semantics. Every is *undecided within budget with no counterexample* after exhausted escalation (smt 14400s, per-method and leg splits, 20-solver portfolio). The residue concentrates in one arithmetic class — slash × ceil-burn `mulDiv` chains on the `take/withdraw` paths; the documented follow-up is `mulDiv` axiomatization. Where a per-method split ran, the proven/undecided method partition is recorded in the run report Notes.

Premise-chain caveat. VS-MI-01 `creditCoversPendingFee` serves as a `requireInvariant` premise in the valid-state setup. It is proven on every non-take method; its take/many legs are (no counterexample) — downstream proofs that consume the premise chain are conditional on those legs.

Setup and Execution

The Certora Prover can be run either remotely (using Certora's cloud infrastructure) or locally (building from source). Both modes share the same initial setup steps.

Common Setup (Steps 1-5)

The instructions below are for Ubuntu 24.04. For step-by-step installation details refer to this setup [tutorial](#).

1. Install Java (tested with JDK 21)

```
sudo apt update
sudo apt install default-jre
java -version
```

2. Install [pipx](#) -- installs Python CLI tools in isolated environments, avoiding dependency conflicts

```
sudo apt install pipx
pipx ensurepath
```

3. Install Certora CLI. To match this report's prover version, pin it explicitly

```
pipx install certora-cli==8.13.0
```

4. Install solc-select and the Solidity compiler version required by the project

```
pipx install solc-select
solc-select install 0.8.34
solc-select use 0.8.34
```

5. Create a versioned solc symlink for Certora. Configuration files reference the compiler as `solc0.8.34` (without dashes), but solc-select only creates a generic `solc` binary. Create the symlink so Certora can find it:

```
mkdir -p ~/.local/bin
ln -sf ~/.solc-select/artifacts/solc-0.8.34/solc-0.8.34 ~/.local/bin/solc0.8.34
```

Verify `~/.local/bin` is in your `PATH`. If not, add it:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Remote Execution

6. Set up Certora key. You can get a free key through the Certora [Discord](#) or on their website. Once you have it, export it:

```
echo "export CERTORAKEY=<your_certora_api_key>" >> ~/.bashrc
```

Note: If a local prover is installed (see below), it takes priority. To force remote execution, add the `--server` `production` flag:

```
certoraRun certora/confs/valid_state_one.conf --server production
```

Local Execution

Follow the full build instructions in the [CertoraProver repository \(v8.13.0\)](#). Once the local prover is installed it takes priority over the remote cloud by default. Tested on Ubuntu 24.04.

1. Install prerequisites

```
# JDK 19+
sudo apt install openjdk-21-jdk

# SMT solvers (z3 and cvc5 are required, others are optional)
# Download binaries and place them in PATH:
#   z3:    https://github.com/Z3Prover/z3/releases
#   cvc5:  https://github.com/cvc5/cvc5/releases

# LLVM tools
sudo apt install llvm

# Rust 1.81.0+
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
cargo install rustfilt

# Graphviz (optional, for visual reports)
sudo apt install graphviz
```

2. Set up build output directory

```
export CERTORA="$HOME/CertoraProver/target/installed/"
mkdir -p "$CERTORA"
export PATH="$CERTORA:$PATH"
```

3. Clone and build

```
git clone --recurse-submodules https://github.com/Certora/CertoraProver.git
cd CertoraProver
git checkout tags/8.13.0
./gradlew assemble
```

4. Verify installation with test example

```
certoraRun.py -h
cd Public/TestEVM/Counter
certoraRun counter.conf
```

Running Verification

The valid-state surface is split into three one-market conf files (no-take, take-only, extended-SMT) and one many-market conf file. The split matches the verification strategy: heavy parametric runs (`take`) get their own conf so the no-take leg can complete quickly without dragging in the heaviest single-method path, and the extended-SMT conf is reserved as the retry target when the default solver budget is insufficient.

Midnight -- Valid State (one-market, no-take)

Runs every parametric method except `take` against the full one-market invariant set:

```
certoraRun certora/confs/valid_state_one.conf
```

Midnight -- Valid State (one-market, take)

Runs only `take` against the full one-market invariant set:

```
certoraRun certora/confs/valid_state_one_take.conf
```

Midnight -- Valid State (one-market, extended SMT)

Same parametric scope as `valid_state_one.conf` but with an extended `smt_timeout` plus a fan-out across `z3` / `cvc5` / `yices` / `bitwuzla` and `split-parallel` mode; the retry target when the default config times out:

```
certoraRun certora/confs/valid_state_one_ext.conf
```

Midnight -- Valid State (many-market)

Runs every parametric method against the many-market invariant set — the valid-state set VS-MI-01..21 plus the cross-market rules HL-MI-22m and ST-MI-17 under the three-market narrowing (`idA`, `idB`, `idC`):

```
certoraRun certora/confs/valid_state_many.conf
```

Midnight -- State Transitions (one-market)

Runs the 18 state-transition rules (ST-MI-01..16, 18, 19; ST-MI-17 is the many-regime cross-market frame) under the one-market narrowing with the callbacks summary loaded:

```
certoraRun certora/confs/state_transition_one.conf
```

Midnight -- High-Level (one-market)

Runs the light high-level tier (HL-MI-01..22 plus the light half of HL-MI-23..62) under the one-market regime with the callbacks summary loaded:

```
certoraRun certora/confs/high_level.conf
```

Midnight -- High-Level heavy tier (one-market, extended SMT)

Runs the heavy nonlinear half of HL-MI-23..62 (slash/lossFactor equations, take pricing exactness, inductive solvency) with the extended smt budget and solver portfolio:

```
certoraRun certora/confs/high_level_heavy.conf
```

Midnight -- Market creation (touchMarket unsummarized)

Runs MC-MI-01..07 against the live creation branch (the only conf without the touchMarket summary). The diagnostic twin `confs/debug/touch_market_diag.conf` is EXPECTED to fail its satisfy in the summarized regime:

```
certoraRun certora/confs/market_creation.conf
```

Midnight -- Reachability (one-market)

Runs the 18 reachability `satisfy()` rules (RC-MI-01..18). A satisfied rule witnesses a reachable path; an UNSAT rule flags a vacuity hole:

```
certoraRun certora/confs/reachability.conf
```

Midnight -- Reverts (one-market)

Runs the 28 revert-condition `@withrevert` rules (RV-MI-01..28) — access-control, input validation, governance enable-gates, and state-precondition guards:

```
certoraRun certora/confs/reverts.conf
```

Midnight -- Gates (one-market)

Enter-gate / liquidator-gate / ratifier enforcement (GT-MI-01..04):

```
certoraRun certora/confs/gates.conf
```

Midnight -- Access Control (one-market)

Runs the 13 parametric access-control rules (AC-MI-01..13) — role-gated config, governance enable-gates, and the authorization graph (state-change $\Rightarrow$ role):

```
certoraRun certora/confs/access_control.conf
```

Resources

- [Certora Tutorials](#) — Official Certora documentation and guided tutorials
- [AlexZoid FV Resources](#) - Curated collection of formal verification resources, examples, and references
- [Updraft Assembly & Formal Verification Course](#) — Comprehensive video course covering assembly and formal verification from the ground up
- [RareSkills Certora Book](#) — Structured tutorial covering CVL syntax, patterns, and common pitfalls